

Chapter 1

Introduction

HTML5 is the **latest** version of HTML, the markup language used to build structure or content on World Wide Web. **HTML5** is the latest web standard by W3C. Earlier HTML was used only to build webpage structure, but **HTML5** is a full-on package to build websites, handle presentation, add functionalities and Web APIs.

HTML5 is the recommended version to build modern web applications as it is light and **fastest** version of **Hypertext Markup Language**, the code that describes web pages. **HTML5** was formed in 2009 and start supporting in browsers in 2010 onwards.

HTML5 Stack

Html5 is the combination of three web technologies:→

HTML → to build webpage structure,

CSS → to enhance look and feel(presentation layer), and

Javascript → to add functionality to HTML elements.

The term **HTML5** means not only HTML, but it is also a combination of HTML, CSS and JavaScript with APIs. For example, drawing and animation using **canvas**, **offline storage**, **microdata**, **audio and video**, **drag and drop**, **geolocation**, **embedded fonts**, web APIs etc. **HTML5** includes new semantic tags and some old tags(*with redefinition*).

HTML + CSS + JAVASCRIPT = HTML5

Why HTML5

HTML5 has been designed to deliver almost everything you'd want to do on web without requiring additional software such as browser plugins. It does everything from **animation** to **apps**, **music** to **movies**, and can also be used to build complicated **web applications** that run in your browser.

HTML5 isn't proprietary, so you don't need to pay royalties to use it. It's also **cross-platform**, which means it doesn't care whether you're using a **tablet** or a **smart phone**, a **netbook**, **notebook**, **ultra-book** or a **Smart TV** if your browser supports **HTML5**, it should work flawlessly.

HTML Versions

Year	Achievement
1989	HTML Formed
1995	HTML 2
1995	CSS
1995	JAVASCRIPT
1997	HTML 4
1998	CSS 2
2000	XHTML 1
2001	XHTML 1.1
2009	HTML5
2015	HTML5.1
2017	HTML5.2
2021	HTML5.3

HTML5 Browsers Support

HTML5 is supported by all major browsers now. The first browser support of html5 was seen in 2010. Here is the list of supported browsers for html5.

- Chrome 5 and above
- Firefox 4 and above
- Opera 11 and latest versions
- Safari 5.1 for Mac, iPhone and iPad
- Internet Explorer 9 and above
- Edge 12 and above Browsers

HTML5 Doctype

HTML5 Doctype is short, and easy to remember. Unlike HTML4 and XHTML, there is no DTD.

```
<!DOCTYPE HTML>    // New and shorter Doctype, no DTD Required
```

HTML5 Template Example

Default Template for HTML5 based websites.

```
<!doctype html>

<html lang="en">      //lang embedded in html

<head>

<title>HTML5 Page</title>

<meta charset="utf-8">    //Short charset meta tag

<link href="style.css" rel="stylesheet"> //No Type attribute

<script src="file.js"></script>    //No Type attribute required

</head>

<body>

    <h1>My First HTML5 page.</h1>

</body>

</html>
```

HTML Elements

An HTML file is made of elements. These elements are responsible for creating web pages and define content in that webpage. An element in HTML usually consist of a start tag <tag name>, close tag </tag name> and content inserted between them. **Technically, an element is a collection of start tag, attributes, end tag, content between them.**

Note: Some elements does not have end tag and content, these elements are termed as empty elements or self-closing element or void elements.

Such as:

```
<p> Hello world!!! </p>
```

Example

```
<!DOCTYPE html>

<html>

<head>

  <title>WebPage</title>

</head>

<body>

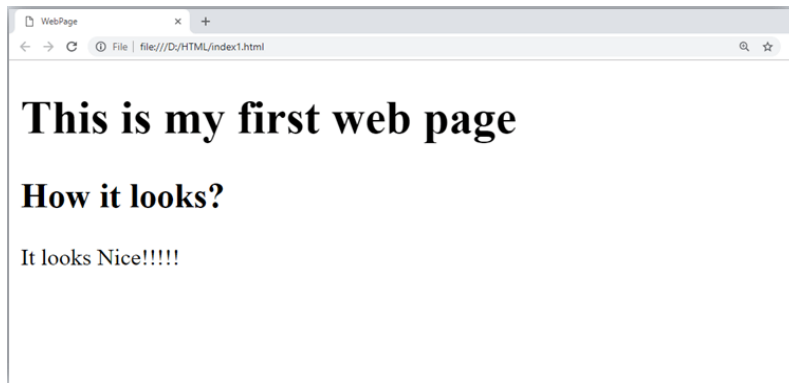
  <h1>This is my first web page</h1>

  <h2> How it looks?</h2>

  <p>It looks Nice!!!!</p>

</body>

</html>
```



Void element: All the elements in HTML do not require to have start tag and end tag, some elements does not have content and end tag such elements are known as Void elements or empty elements. **These elements are also called unpaired tags.**

Some Void elements are
 (represents a line break) , <hr>(represents a horizontal line), etc.

Nested HTML Elements: HTML can be nested, which means an element can contain another element.

Block-level and Inline HTML elements

For the default display and styling purpose in HTML, all the elements are divided into two categories:

- Block-level element
- Inline element

Block-level element:

- These are the elements, which structure main part of web page, by dividing a page into coherent blocks.
- A block-level element always start with new line and takes the full width of web page, from left to right.
- These elements can contain block-level as well as inline elements.

Following are the block-level elements in HTML.

<address>, <article>, <aside>, <blockquote>, <canvas>, <dd>, <div>, <dl>, <dt>, <fieldset>, <figcaption>, <figure>, <footer>, <form>, <h1>-<h6>, <header>, <hr>, , <main>, <nav>, <noscript>, , <output>, <p>, <pre>, <section>, <table>, <tfoot>, and <video>.

Example:

1. <!DOCTYPE html>

<html>

<head>

</head>

<body>

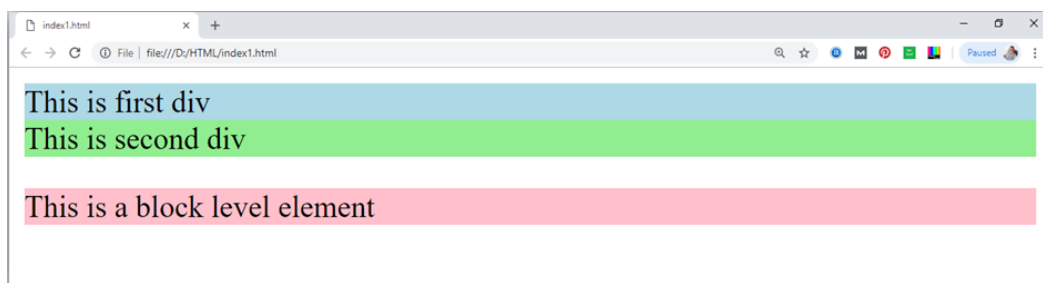
<div style="background-color: lightblue">This is first div</div>

<div style="background-color: lightgreen">This is second div</div>

<p style="background-color: pink">This is a block level element</p>

</body>

</html>



Inline elements:

- Inline elements are those elements, which differentiate the part of a given text and provide it a particular function.
- These elements does not start with new line and take width as per requirement.
- The Inline elements are mostly used with other elements.

<a>, <abbr>, <acronym>, , <bdo>, <big>,
, <button>, <cite>, <code>, <dfn>, , <i>, , <input>, <kbd>, <label>, <map>, <object>, <q>, <samp>, <script>, <select>, <small>, , , <sub>, <sup>, <textarea>, <time>, <tt>, <var>.

Example:

```
<!DOCTYPE html>

<html>

  <head>

  </head>

  <body>

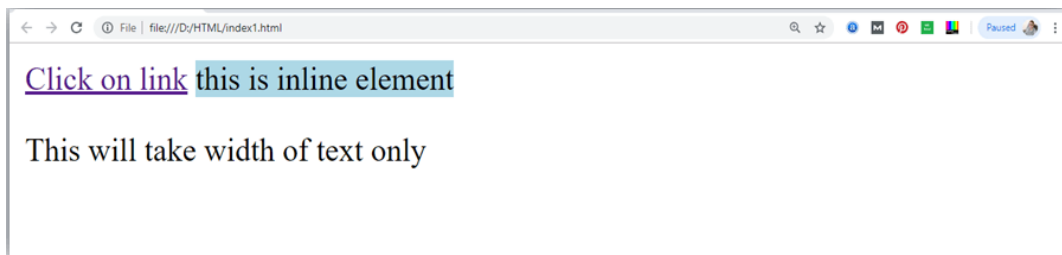
    <a href="https://www.google.com">Click on link</a>

    <span style="background-color: lightblue">this is inline element</span>

    <p>This will take width of text only</p>

  </body>

</html>
```



Following is the list of the some main elements used in HTML:

Start tag	Content	End tag	Description
<h1> <h6>	These are headings of HTML	</h1>??..</h6>	These elements are used to provide the headings of page.
<p>	This is the paragraph	</p>	This element is used to display a content in form of paragraph.
<div>	This is div section	</div>	This element is used to provide a section in web page.

			This element is used to provide a line break. (void element)
<hr>			This element is used to provide a horizontal line. (void element)

HTML Formatting

HTML Formatting is a process of formatting text for better look and feel. HTML provides us ability to format text without using CSS. There are many formatting tags in HTML. These tags are used to make text bold, italicized, or underlined. There are almost 14 options available that how text appears in HTML and XHTML.

In HTML the formatting tags are divided into two categories:

- Physical tag: These tags are used to provide the visual appearance to the text.
- Logical tag: These tags are used to add some logical or semantic value to the text.
- Following is the list of HTML formatting text.

Element name	Description
	This is a physical tag, which is used to bold the text written between it.
	This is a logical tag, which tells the browser that the text is important.
<i>	This is a physical tag which is used to make text italic.
	This is a logical tag which is used to display content in italic.
<mark>	This tag is used to highlight text.
<u>	This tag is used to underline text written between it.
<tt>	This tag is used to appear a text in teletype. (not supported in HTML5)
<strike>	This tag is used to draw a strikethrough on a section of text. (Not supported

	in HTML5)
<sup>	It displays the content slightly above the normal line.
<sub>	It displays the content slightly below the normal line.
	This tag is used to display the deleted content.
<ins>	This tag displays the content which is added
<big>	This tag is used to increase the font size by one conventional unit.
<small>	This tag is used to decrease the font size by one unit from base font size.

Example:

```

<!DOCTYPE html>

<html>

<head><title> Text Formatting Tags </title>

</head>

<body>

<h1> Formatting Tags </h1>

<p><b> This text is bold </b></p>

<p><i> This text is italic </i></p>

<p> This is <sub> subscript </sub> and <sup> superscript </sup> </p>

<p> <strong> This text is strong </strong> </p>

<p><em> This text is emphasized. </em></p>

<h2> HTML <small> Small </small> Formatting </h2>

<h2> HTML <mark> Marked </mark> Formatting </h2>

<p> The del element represents deleted(removed)text. </p>

<p> My favourite color is <del>blue</del> red. </p>

<p> My favourite <ins> color</ins> is red. </p>

</body>

</html>

```


OUTPUT:

Formatting Tags

This text is bold

This text is italic

This is _{subscript} and ^{superscript}

This text is strong

This text is emphasized.

HTML Small Formatting

HTML **Marked** Formatting

The del element represents deleted(removed)text.

My favourite color is ~~blue~~ red.

My favourite color is red.

HTML Heading

A HTML heading or HTML h tag can be defined as a title or a subtitle which you want to display on the webpage. When you place the text within the heading tags <h1>.....</h1>, it is displayed on the browser in the bold format and size of the text depends on the number of heading.

There are six different HTML headings which are defined with the <h1> to <h6> tags, from highest level h1 (main heading) to the least level h6 (least important heading).

h1 is the largest heading tag and h6 is the smallest one. So h1 is used for most important heading and h6 is used for least important.

Headings in HTML helps the search engine to understand and index the structure of web page.

Example:

<h1>Heading no. 1</h1>

<h2>Heading no. 2</h2>

<h3>Heading no. 3</h3>

<h4>Heading no. 4</h4>

<h5>Heading no. 5</h5>

<h6>Heading no. 6</h6>

Output:

Heading no. 1

Heading no. 2

Heading no. 3

Heading no. 4

Heading no. 5

Heading no. 6

HTML Anchor

The **HTML anchor tag** defines *a hyperlink that links one page to another page*. It can create hyperlink to other web page as well as files, location, or any URL. The "href" attribute is the most important attribute of the HTML a tag. and which links to destination page or URL.

href attribute of HTML anchor tag

The href attribute is used to define the address of the file to be linked. In other words, it points out the destination page.

The syntax of HTML anchor tag is given below.

```
<a href = "....."> Link Text </a>
```

Let's see an example of HTML anchor tag.

```
<a href="second.html">Click for Second Page</a>
```

Specify a location for Link using target attribute

If we want to open that link to another page then we can use target attribute of <a> tag. With the help of this link will be open in next page.

Example:

```
<!DOCTYPE html>

<html>

<head>

  <title></title>

</head>

<body>

  <p>Click on <a href="https://www.google.com/" target="_blank"> this-
  link </a>to go on home page of Google</p>

</body>

</html>
```

- The **target** attribute can only use with href attribute in anchor tag.
- If we will not use target attribute then link will open in same page.

Appearance of HTML anchor tag

An **unvisited link** is displayed underlined and blue.

A **visited link** displayed underlined and purple.

An **active link** is underlined and red.

HTML Comments

Comments are some text or code written in your code to give an explanation about the code, and not visible to the user. Comments which are used for HTML file are known as HTML comments. Anything written between these tags will be ignored by the browser, so comments will not be visible on the webpage.

Comments of any code make code easy to understand and increase readability of code.

Comments are also part of the code, which explains the code.

How to add comment In HTML

You can add comments in your HTML file using `<!-- ... -->` tag. So if you will write anything between these comment tags that will be treated as comment and browser will not read it.

Syntax

```
<!-- Write commented text here -->
```

Note: The commented code will not be visible to a webpage, and hence you can use comment tags for documentation purpose, and debugging purpose:

Such as:

```
<!-- <p>There is some text</p>
      <p>There is second text</p> -->
```

Example:

```
<!DOCTYPE html>
<html>
<!-- This is Header section -->
<head>
  <!-- Internal CSS -->
  <style>
    body{
      text-align: center;
      background-color: #f0f8ff;
      font-size: 30px;
      color: red;
    }
  </style>
</head>
```

```
<!-- This is body section, write code here which you want to display on web-page -->

<body>

  <!-- heading tag -->

  <h2>First WebPage</h2>


  <!-- Paragraph tag -->

  <p>Write your Content here!!!</p>

</body>

</html>
```

Multiline Comment

In HTML code, we can also comments multiple lines at a time. In multiline comment we can use any description about code or multiple line code to debug, etc.

Syntax

```
<!---

Your code is commented.

Write description of code.

It will not display at webpage.

-->
```

HTML Image

HTML img tag is used to display image on the web page. HTML img tag is an empty tag that contains attributes only, closing tags are not used in HTML image element.

Let's see an example of HTML image.

```
<h2>HTML Image Example</h2>


```

Attributes of HTML img tag

The src and alt are important attributes of HTML img tag. All attributes of HTML image tag are given below.

1) src

It is a necessary attribute that describes the source or path of the image. It instructs the browser where to look for the image on the server.

The location of image may be on the same directory or another server.

2) alt

The alt attribute defines an alternate text for the image, if it can't be displayed. The value of the alt attribute describe the image in words. The alt attribute is considered good for SEO prospective.

3) width

It is an optional attribute which is used to specify the width to display the image. It is not recommended now. You should apply CSS in place of width attribute.

4) height

It h3 the height of the image. The HTML height attribute also supports iframe, image and object elements. It is not recommended now. You should apply CSS in place of height attribute.

Use of height and width attribute with img tag

You have learnt about how to insert an image in your web page, now if we want to give some height and width to display image according to our requirement, then we can set it with height and width attributes of image.

Example:

```

```

Use of alt attribute

We can use alt attribute with `img` tag. It will display an alternative text in case if image cannot be displayed on browser. Following is the example for alt attribute:

```

```

How to get image from another directory/folder?

To insert an image in your web, that image must be present in your same folder where you have put the HTML file. But if in some case image is available in some other directory then you can access the image like this:

```

```

Use tag as a link

We can also link an image with other page or we can use an image as a link. To do this, put tag inside the <a> tag.

Example:

```
<a href="https://www.Google.com"></a>
```

HTML Table

HTML table tag is used to display data in tabular form (row * column). There can be many columns in a row.

We can create a table to display data in tabular form, using <table> element, with the help of <tr>, <td>, and <th> elements.

In Each table, table row is defined by <tr> tag, table header is defined by <th>, and table data is defined by <td> tags.

HTML tables are used to manage the layout of the page e.g. header section, navigation bar, body content, footer section etc. But it is recommended to use div tag over table to manage the layout of the page .

HTML Table Tags

Tag	Description
<table>	It defines a table.
<tr>	It defines a row in a table.
<th>	It defines a header cell in a table.
<td>	It defines a cell in a table.
<caption>	It defines the table caption.

<colgroup>	It specifies a group of one or more columns in a table for formatting.
<col>	It is used with <colgroup> element to specify column properties for each column.
<tbody>	It is used to group the body content in a table.
<thead>	It is used to group the header content in a table.
<tfooter>	It is used to group the footer content in a table.

HTML Table Example

Let's see the example of HTML table tag. Its output is shown above.

```
<table>

<tr><th>First_Name</th><th>Last_Name</th><th>Marks</th></tr>

<tr><td>Sonoo</td><td>Jaiswal</td><td>60</td></tr>

<tr><td>James</td><td>William</td><td>80</td></tr>

<tr><td>Swati</td><td>Sironi</td><td>82</td></tr>

<tr><td>Chetna</td><td>Singh</td><td>72</td></tr>

</table>
```

HTML Table with Border

There are two ways to specify border for HTML tables.

1. By border attribute of table in HTML
2. By border property in CSS

Example:

```
<table border="1">

<tr><th>First_Name</th><th>Last_Name</th><th>Marks</th></tr>

<tr><td>Sonoo</td><td>Jaiswal</td><td>60</td></tr>

<tr><td>James</td><td>William</td><td>80</td></tr>
```



```
<tr><td>Swati</td><td>Sironi</td><td>82</td></tr>
<tr><td>Chetna</td><td>Singh</td><td>72</td></tr>
</table>
```

CSS Border property

It is now recommended to use border property of CSS to specify border in table.

```
<style>
table, th, td {
    border: 1px solid black;
}
</style>
```

HTML Table with cell padding

You can specify padding for table header and table data by two ways:

By cellpadding attribute of table in HTML

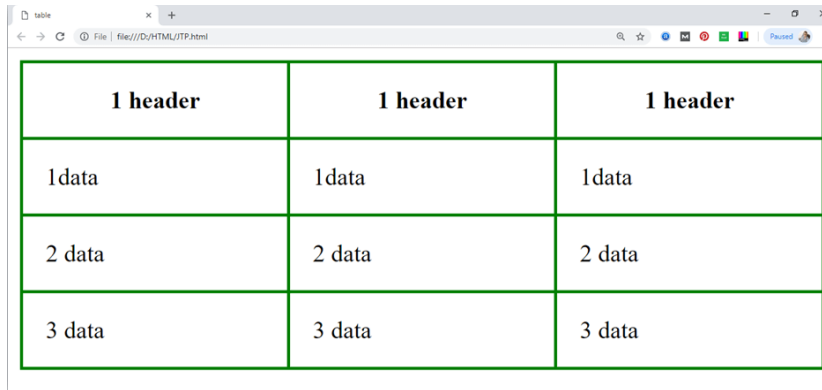
By padding property in CSS

```
<style>
table, th, td {
    border: 1px solid pink;
    border-collapse: collapse;
}
th, td {
    padding: 10px;
}
</style>
```

HTML Table width:

We can specify the HTML table width using the **CSS width** property. It can be specify in pixels or percentage.

```
<!DOCTYPE html>
<html>
<head>
  <title>table</title>
  <style>
    table{
      border-collapse: collapse;
      width: 100%;
    }
    th,td{
      border: 2px solid green;
      padding: 15px;
    }
  </style>
</head>
<body>
  <table>
    <tr>
      <th>1 header</th>
      <th>1 header</th>
      <th>1 header</th>
    </tr>
    <tr>
      <td>1data</td>
      <td>1data</td>
      <td>1data</td>
    </tr>
    <tr>
      <td>2 data</td>
      <td>2 data</td>
      <td>2 data</td>
    </tr>
    <tr>
      <td>3 data</td>
      <td>3 data</td>
      <td>3 data</td>
    </tr>
  </table>
</body>
</html>
```



1 header	1 header	1 header
1 data	1 data	1 data
2 data	2 data	2 data
3 data	3 data	3 data

HTML Table with colspan

It will divide one cell/row into multiple columns, and the number of columns depend on the value of colspan attribute.

CSS code:

```
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
th, td {
  padding: 5px;
}
</style>
```

HTML code:

```
<table style="width:100%">

<tr>

<th>Name</th>

<th colspan="2">Mobile No.</th>

</tr>

<tr>

<td>Ajeet Maurya</td>

<td>7503520801</td>

<td>9555879135</td>
```

```
</tr>
</table>
```

HTML Table with rowspan

If you want to make a cell span more than one row, you can use the rowspan attribute.

It will divide a cell into multiple rows. The number of divided rows will depend on rowspan values.

CSS code:

```
<style>
table, th, td {
    border: 1px solid black;
    border-collapse: collapse;
}
th, td {
    padding: 10px;
}
</style>
```

HTML code:

```
<table>
<tr><th>Name</th><td>Ajeet Maurya</td></tr>
<tr><th rowspan="2">Mobile No.</th><td>7503520801</td></tr>
<tr><td>9555879135</td></tr>
</table>
```

HTML table with caption

```
<table>
<caption>Student Records</caption>
```

```
<tr><th>First_Name</th><th>Last_Name</th><th>Marks</th></tr>
<tr><td>Vimal</td><td>Jaiswal</td><td>70</td></tr>
<tr><td>Mike</td><td>Warn</td><td>60</td></tr>
<tr><td>Shane</td><td>Warn</td><td>42</td></tr>
<tr><td>Jai</td><td>Malhotra</td><td>62</td></tr>
</table>
```

HTML Lists

HTML Lists are used to specify lists of information. All lists may contain one or more list elements. There are three different types of HTML lists:

1. Ordered List or Numbered List (ol)
2. Unordered List or Bulleted List (ul)
3. Description List or Definition List (dl)

HTML Ordered List or Numbered List

In the ordered HTML lists, all the list items are marked with numbers by default. It is known as numbered list also. The ordered list starts with tag and the list items start with tag.

```
<ol>
<li>Aries</li>
<li>Bingo</li>
<li>Leo</li>
<li>Oracle</li>
</ol>
```

HTML Unordered List or Bulleted List

In HTML Unordered list, all the list items are marked with bullets. It is also known as bulleted list also. The Unordered list starts with tag and list items start with the tag.

```
<ul>
<li>Aries</li>
```

```
<li>Bingo</li>
<li>Leo</li>
<li>Oracle</li>
</ul>
```

HTML Description List or Definition List

HTML Description list is also a list style which is supported by HTML and XHTML. It is also known as definition list where entries are listed like a dictionary or encyclopedia.

The definition list is very appropriate when you want to present glossary, list of terms or other name-value list.

The HTML definition list contains following three tags:

<dl> tag defines the start of the list.

<dt> tag defines a term.

<dd> tag defines the term definition (description).

<dl>

<dt>Aries**</dt>**

<dd>-One of the 12 horoscope sign.**</dd>**

<dt>Bingo**</dt>**

<dd>-One of my evening snacks**</dd>**

<dt>Leo**</dt>**

<dd>-It is also an one of the 12 horoscope sign.**</dd>**

<dt>Oracle**</dt>**

<dd>-It is a multinational technology corporation.**</dd>**

</dl>

HTML Nested List

A list within another list is termed as nested list. If you want a bullet list inside a numbered list then such type of list will be called as nested list.

```
<!DOCTYPE html>
<html>
<head>
  <title>Nested list</title>
</head>
<body>
  <p>List of Indian States with their capital</p>
<ol>
  <li>Delhi
    <ul>
      <li>New Delhi</li>
    </ul>
  </li>
  <li>Haryana
    <ul>
      <li>Chandigarh</li>
    </ul>
  </li>
  <li>Gujarat
    <ul>
      <li>Gandhinagar</li>
    </ul>
  </li>
  <li>Rajasthan
    <ul>
      <li>Jaipur</li>
    </ul>
  </li>
  <li>Maharashtra
    <ul>
      <li>Mumbai</li>
    </ul>
  </li>
  <li>Uttarpradesh
    <ul>
      <li>Lucknow</li></ul>
  </li>
</ol>
</body>
</html>
```

Output:

List of Indian States with thier capital

1. Delhi
 - NewDelhi
2. Haryana
 - Chandigarh
3. Gujarat
 - Gandhinagar
4. Rajasthan
 - Jaipur
5. Maharashtra
 - Mumbai
6. Uttarpradesh
 - Lucknow

Chapter 2

HTML Form

An **HTML form** is a section of a document which contains controls such as text fields, password fields, checkboxes, radio buttons, submit button, menus etc.

An HTML form facilitates the user to enter data that is to be sent to the server for processing such as name, email address, password, phone number, etc. .

Why use HTML Form

HTML forms are required if you want to collect some data from of the site visitor.

For example: If a user want to purchase some items on internet, he/she must fill the form such as shipping address and credit/debit card details so that item can be sent to the given address.

HTML Form Syntax

```
<form action="server url" method="get|post">
```

```
//input controls e.g. textfield, textarea, radiobutton, button
```

```
</form>
```

HTML Form Tags

Let's see the list of HTML 5 form tags.

Tag	Description
<form>	It defines an HTML form to enter inputs by the used side.
<input>	It defines an input control.
<textarea>	It defines a multi-line input control.
<label>	It defines a label for an input element.
<fieldset>	It groups the related element in a form.
<legend>	It defines a caption for a <fieldset> element.
<select>	It defines a drop-down list.
<optgroup>	It defines a group of related options in a drop-down list.
<option>	It defines an option in a drop-down list.

<button>	It defines a clickable button.
<datalist>	It specifies a list of pre-defined options for input control.
<keygen>	It defines a key-pair generator field for forms.
<output>	It defines the result of a calculation.

HTML <form> element

The HTML <form> element provide a document section to take input from user. It provides various interactive controls for submitting information to web server such as text field, text area, password field, etc.

Syntax:

<form>

//Form elements

</form>

HTML <input> element

The HTML <input> element is fundamental form element. It is used to create form fields, to take input from user. We can apply different input filed to gather different information form user. Following is the example to show the simple text input.

Example:

<body>

<form>

Enter your name **
**

<input type="text" name="username">

</form>

</body>

Enter your name

HTML TextField Control

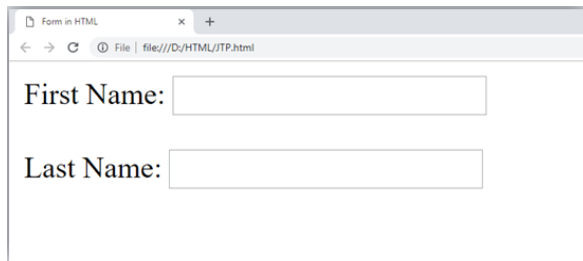
The type="text" attribute of input tag creates textfield control also known as single line textfield control. The name attribute is optional, but it is required for the server side component such as JSP, ASP, PHP etc.

<form>

First Name: **<input type="text" name="firstname"/>
**

Last Name: **<input type="text" name="lastname"/>
**

</form>

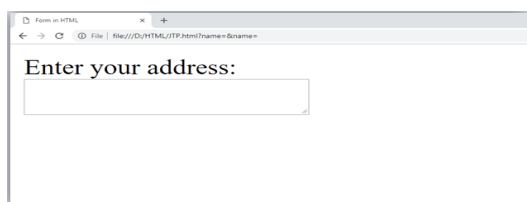
A screenshot of a web browser window titled 'Form in HTML'. The address bar shows 'file:///D:/HTML/TP.html'. The page content displays two text input fields. The first field is preceded by the label 'First Name:' and the second field is preceded by 'Last Name:'. Both fields are empty.

HTML <textarea> tag in form

The <textarea> tag in HTML is used to insert multiple-line text in a form. The size of <textarea> can be specify either using "rows" or "cols" attribute or by CSS.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Form in HTML</title>
</head>
<body>
  <form>
    Enter your address:<br>
    <textarea rows="2" cols="20"></textarea>
  </form>
</body>
</html>
```

A screenshot of a web browser window titled 'Form in HTML'. The address bar shows 'file:///D:/HTML/TP.html?name=&name='. The page content displays a label 'Enter your address:' followed by a multi-line text area with two visible rows. The text area is empty.

Label Tag in Form

It is considered better to have label in form. As it makes the code parser/browser/user friendly.

If you click on the label tag, it will focus on the text control. To do so, you need to have for attribute in label tag that must be same as id attribute of input tag.

It is good to use <label> tag with form, although it is optional but if you will use it, then it will provide a focus when you tap or click on label tag. It is more worthy with touchscreens.

```
<form>
```

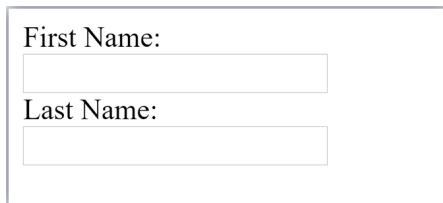
```
<label for="firstname">First Name: </label> <br/>
```

```
<input type="text" id="firstname" name="firstname"/> <br/>
```

```
<label for="lastname">Last Name: </label>
```

```
<input type="text" id="lastname" name="lastname"/> <br/>
```

```
</form>
```



First Name:

Last Name:

HTML Password Field Control

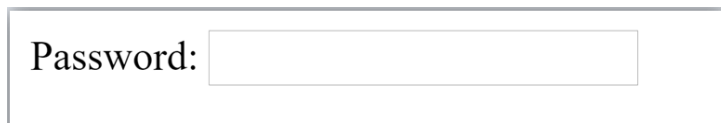
The password is not visible to the user in password field control.

```
<form>
```

```
<label for="password">Password: </label>
```

```
<input type="password" id="password" name="password"/> <br/>
```

```
</form>
```



Password:

HTML 5 Email Field Control

The email field is new in HTML 5. It validates the text for correct email address. You must use @ and . in this field.

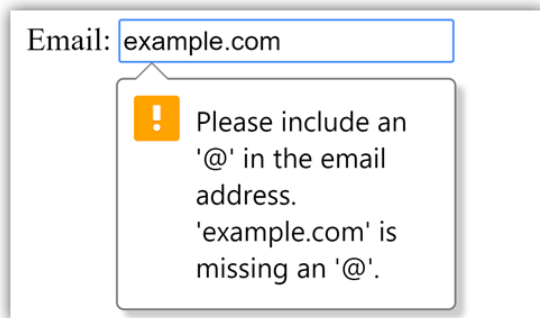
```
<form>
```

```
  <label for="email">Email: </label>
```

```
    <input type="email" id="email" name="email"/> <br/>
```

```
</form>
```

Email:



Radio Button Control

The radio button is used to select one option from multiple options. It is used for selection of gender, quiz questions etc.

If you use one name for all the radio buttons, only one radio button can be selected at a time.

Using radio buttons for multiple options, you can only choose a single option at a time.

```
<form>
```

```
  <label for="gender">Gender: </label>
```

```
    <input type="radio" id="gender" name="gender" value="male"/>Male
```

```
    <input type="radio" id="gender" name="gender" value="female"/>Female <br/>
```

```
</form>
```

Gender: ☐ Male ☒ Female

Checkbox Control

The checkbox control is used to check multiple options from given checkboxes.

<form>

Hobby:**
**

<input type="checkbox" id="cricket" name="cricket" value="cricket"/>

**<label for="cricket">Cricket</label>
**

<input type="checkbox" id="football" name="football" value="football"/>

**<label for="football">Football</label>
**

<input type="checkbox" id="hockey" name="hockey" value="hockey"/>

<label for="hockey">Hockey</label>

</form>

These are similar to radio button except it can choose multiple options at a time and radio button can select one button at a time, and its display.

Hobby:

☒ Cricket

☒ Football

☐ Hockey

Submit button control

HTML **<input type="submit">** are used to add a submit button on web page. When user clicks on submit button, then form get submit to the server.

Syntax:

<input type="submit" value="submit">

The type = submit , specifying that it is a submit button

The value attribute can be anything which we write on button on web page.

The name attribute can be omit here.

Example:

```
<form>

  <label for="name">Enter name</label><br>

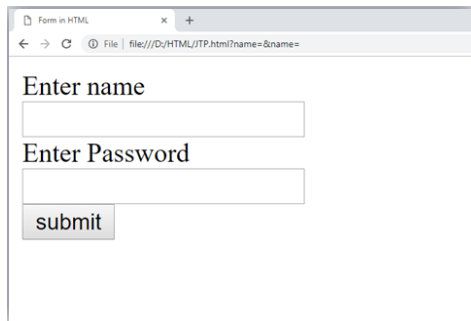
  <input type="text" id="name" name="name"><br>

  <label for="pass">Enter Password</label><br>

  <input type="Password" id="pass" name="pass"><br>

  <input type="submit" value="submit">

</form>
```

A screenshot of a web browser window titled 'Form in HTML'. The address bar shows a file path: 'file:///D:/HTML/JTP.html?name=&name='. The form contains two text input fields. The first field is labeled 'Enter name' and the second is labeled 'Enter Password'. Below the password field is a 'submit' button.**HTML <fieldset> element:**

The <fieldset> element in HTML is used to group the related information of a form. This element is used with <legend> element which provide caption for the grouped elements.

```
<form>

  <fieldset>

    <legend>User Information:</legend>

    <label for="name">Enter name</label><br>

    <input type="text" id="name" name="name"><br>

    <label for="pass">Enter Password</label><br>

    <input type="Password" id="pass" name="pass"><br>

    <input type="submit" value="submit">

  </fieldset>

</form>
```

User Information:

Enter name

Enter Password

submit

HTML Audio Tag

HTML audio tag is used to define sounds such as music and other audio clips. Currently there are three supported file format for HTML 5 audio tag.

1. mp3
2. wav
3. ogg

HTML5 supports <video> and <audio> controls. The Flash, Silverlight and similar technologies are used to play the multimedia items.

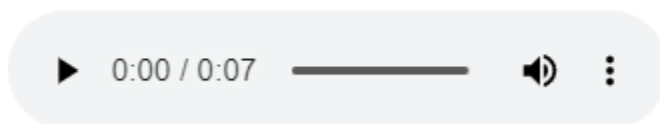
<audio controls>

<source src="koyal.mp3" type="audio/mpeg">

Your browser does not support the html audio tag.

</audio>

Output:



Attributes of HTML Audio Tag

There is given a list of HTML audio tag.

Attribute	Description
controls	It defines the audio controls which is displayed with play/pause buttons.
autoplay	It specifies that the audio will start playing as soon as it is ready.
loop	It specifies that the audio file will start over again, every time when it is completed.
muted	It is used to mute the audio output.
preload	It specifies the author view to upload audio file when the page loads.
src	It specifies the source URL of the audio file.

HTML Audio Tag Attribute Example

Here we are going to use controls, autoplay, loop and src attributes of HTML audio tag.

```
<audio controls autoplay loop>
```

```
<source src="koyal.mp3" type="audio/mpeg"></audio>
```

MIME Types for HTML Audio format

The available MIME type HTML audio tag is given below.

Audio Format	MIME Type
mp3	audio/mpeg
ogg	audio/ogg
wav	audio/wav

HTML Video Tag

HTML 5 supports <video> tag also. The HTML video tag is used for streaming video files such as a movie clip, song clip on the web page.

Currently, there are three video formats supported for HTML video tag:

1. mp4
2. webM
3. ogg

HTML Video Tag Example

<video controls>

<source src="movie.mp4" type="video/mp4">

Your browser does not support the html video tag.

</video>

Attributes of HTML Video Tag

Attribute	Description
controls	It defines the video controls which is displayed with play/pause buttons.
height	It is used to set the height of the video player.
width	It is used to set the width of the video player.
poster	It specifies the image which is displayed on the screen when the video is not played.
autoplay	It specifies that the video will start playing as soon as it is ready.
loop	It specifies that the video file will start over again, every time when it is completed.
muted	It is used to mute the video output.
preload	It specifies the author view to upload video file when the page loads.
src	It specifies the source URL of the video file.

HTML Video Tag Attribute Example

Let's see the example of video tag in HTML where are using height, width, autoplay, controls and loop attributes.

```
<video width="320" height="240" controls autoplay loop>
```

```
<source src="movie.mp4" type="video/mp4">
```

Your browser does not support the html video tag.

```
</video>
```

MIME Types for HTML Video format

The available MIME type HTML video tag is given below.

Video Format	MIME Type
mp4	video/mp4
ogg	video/ogg
webM	video/webM

Chapter 3

CSS

- CSS stands for Cascading Style Sheet.
- CSS is used to design HTML tags.
- CSS is a widely used language on the web.
- HTML, CSS and JavaScript are used for web designing. It helps the web designers to apply style on HTML tags.

CSS Example with CSS Editor

```
<!DOCTYPE>
<html>
<head>
<style>
h1{
color:white;
background-color:red;
padding:5px;
}
p{
color:blue;
}
</style>
</head>
<body>
<h1>Write Your First CSS Example</h1>
<p>This is Paragraph.</p>
</body>
</html>
```

CSS Syntax

The CSS provides the style to the HTML element, which the browser interprets. After being interpreted by the browser, the CSS style property will be applied to all the elements of the HTML. We can provide style property to the HTML element in three parts. These three parts are as follows.

1. Selector

It is an HTML tag. All the style properties of the CSS will be applied to the selector. The selector tag like **<h1>** or **<table>** etc.

2. Property

It is a type of attribute that is present in HTML tags. All the attributes of the HTML will be converted to the CSS properties. The CSS properties like *color*, *border*, etc.

3. Value

In HTML, these are assigned to the properties. For example, *the color* property can have a value of either *red* or *#F1F1F1*, etc.

Syntax:

We must provide the CSS property to the HTML element in a proper way. We must follow the syntax below to implement the CSS property.

```
selector { property: value }
```



Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <style>
    /* styles.css */

    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-color: #f2f2f2;
    }

    #header {
      background-color: #333;
      color: white;
      text-align: center;
```

```
padding: 20px;
}

h1 {
font-size: 36px;
margin-bottom: 20px;
}

p {
font-size: 18px;
margin-bottom: 10px;
}

.button {
background-color: #ff0000;
color: white;
padding: 10px 20px;
border: none;
text-align: center;
text-decoration: none;
display: inline-block;
font-size: 16px;
border-radius: 5px;
cursor: pointer;
}
.button:hover {
background-color: #990000;
}

</style>
</head>
<body>
<div id="header">
<h1>Hello, CSS Example!</h1>
<p>Welcome to MyClass. Here is a styled button:</p>
<button class="button">Click Me!</button>
</div>
</body>
</html>
```

- In the above example, there is a body tag. In the body tag, we set the entire page's font family, margin, padding, and background color.
- Also, there is a #header selector. Inside the #header selector, we have set the header section's background color, text color, text alignment, and padding.

- Inside the h1 selector, we have set the font size and margin for the main heading.
- Inside the p selector, we have set font size and margin for paragraphs.
- We have set the button with a red background, white text color, padding, border, and other properties inside the button selector.
- Inside the .button:hover selector, we have set the background color to change when the button hovers, giving it a darker shade of red.

The CSS element Selector

The element selector selects HTML elements based on the element name.

Example

Here, all <p> elements on the page will be center-aligned, with a red text color:

```
p {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector

The id selector uses the id attribute of an HTML element to select a specific element. The id of an element is unique within a page, so the id selector is used to select one unique element!

To select an element with a specific id, write a hash (#) character, followed by the id of the element.

Example

The CSS rule below will be applied to the HTML element with id="para1":

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

The CSS class Selector

The class selector selects HTML elements with a specific class attribute.

To select elements with a specific class, write a period (.) character, followed by the class name.

Example

In this example all HTML elements with class="center" will be red and center-aligned:

```
.center {  
  text-align: center;  
  color: red;  
}
```

You can also specify that only specific HTML elements should be affected by a class.

Example

In this example only <p> elements with class="center" will be red and center-aligned:

```
p.center {  
  text-align: center;  
  color: red;  
}
```

The CSS Universal Selector

The universal selector (*) selects all HTML elements on the page.

Example

The CSS rule below will affect every HTML element on the page:

```
* {  
  text-align: center;  
  color: blue;  
}
```

The CSS Grouping Selector

The grouping selector selects all the HTML elements with the same style definitions.

Look at the following CSS code (the h1, h2, and p elements have the same style definitions.


```
h1 {  
  text-align: center;  
  color: red;  
}
```

```
h2 {  
  text-align: center;  
  color: red;  
}
```

```
p {  
  text-align: center;  
  color: red;  
}
```

It will be better to group the selectors, to minimize the code.

To group selectors, separate each selector with a comma.

Example

In this example we have grouped the selectors from the code above:

```
h1, h2, p {  
  text-align: center;  
  color: red;  
}
```

All CSS Simple Selectors

Selector	Example	Example description
#id	#firstname	Selects the element with id="firstname"
.class	.intro	Selects all elements with class="intro"
element.class	p.intro	Selects only <p> elements with class="intro"
*	*	Selects all elements
element	p	Selects all <p> elements
element,element,..	div, p	Selects all <div> elements and all <p> elements

How To Add CSS

There are three ways of inserting a style sheet:

- External CSS
- Internal CSS
- Inline CSS

External CSS

- With an external style sheet, you can change the look of an entire website by changing just one file!
- Each HTML page must include a reference to the external style sheet file inside the <link> element, inside the head section.

Example

- External styles are defined within the <link> element, inside the <head> section of an HTML page:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="mystyle.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

An external style sheet can be written in any text editor, and must be saved with a .css extension.

The external .css file should not contain any HTML tags.

Here is how the "mystyle.css" file looks:

```
"mystyle.css"
```

```
body {
  background-color: lightblue;
}
```

```
h1 {
  color: navy;
  margin-left: 20px;
}
```

Note: Do not add a space between the property value (20) and the unit (px):

Incorrect (space): margin-left: 20 px;

Correct (no space): margin-left: 20px;

Internal CSS

An internal style sheet may be used if one single HTML page has a unique style. The internal style is defined inside the <style> element, inside the head section.

Example

Internal styles are defined within the <style> element, inside the <head> section of an HTML page:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {
  background-color: linen;
}

h1 {
  color: maroon;
  margin-left: 40px;
}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Inline CSS

An inline style may be used to apply a unique style for a single element.

To use inline styles, add the style attribute to the relevant element. The style attribute can contain any CSS property.

Example

Inline styles are defined within the "style" attribute of the relevant element:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;text-align:center;">This is a heading</h1>
<p style="color:red;">This is a paragraph.</p>

</body>
</html>
```

Multiple Style Sheets

If some properties have been defined for the same selector (element) in different style sheets, the value from the last read style sheet will be used.

Assume that an **external style sheet** has the following style for the <h1> element:

```
h1 {
  color: navy;
}
```

Then, assume that an **internal style sheet** also has the following style for the <h1> element:

```
h1 {
  color: orange;
}
```

Example

If the internal style is defined **after** the link to the external style sheet, the <h1> elements will be "orange":

```
<head>
<link rel="stylesheet" type="text/css" href="mystyle.css">
<style>
h1 {
  color: orange;
}
</style>
</head>
```

Example

However, if the internal style is defined **before** the link to the external style sheet, the <h1> elements will be "navy":

```
<head>
<style>
h1 {
  color: orange;
}
</style>
<link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
```

CSS Comments

Comments are used to explain the code, and may help when you edit the source code at a later date.

Comments are ignored by browsers.

A CSS comment is placed inside the <style> element, and starts with /* and ends with */:

Example

```
/* This is a single-line comment */
p {
  color: red;
}
```

You can add comments wherever you want in the code:

Example

```
p {
  color: red; /* Set text color to red */
}
```

Even in the middle of a code line:

Example

```
p {
  color: /*red*/blue;
}
```

Comments can also span multiple lines:

Example

```
/* This is
a multi-line
```

```
comment */
```

```
p {  
  color: red;  
}
```

HTML and CSS Comments

From the HTML tutorial, you learned that you can add comments to your HTML source by using the `<!--...-->` syntax.

In the following example, we use a combination of HTML and CSS comments:

Example

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
p {  
  color: red; /* Set text color to red */  
}  
</style>  
</head>  
<body>  
  
<h2>My Heading</h2>  
  
<!-- These paragraphs will be red -->  
<p>Hello World!</p>  
<p>This paragraph is styled with CSS.</p>  
<p>CSS comments are not shown in the output.</p>  
  
</body>  
</html>
```

CSS Colors

Colors are specified using predefined color names, or RGB, HEX, HSL, RGBA, HSLA values.

CSS Color Names

In CSS, a color can be specified by using a predefined color name:

Tomato	Orange	DodgerBlue	MediumSeaGreen
Gray	SlateBlue	Violet	LightGray

CSS Background Color

Example

```
<h1 style="background-color:DodgerBlue;">Hello World</h1>  
<p style="background-color:Tomato;">Lorem ipsum...</p>
```

CSS Text Color

Example

```
<h1 style="color:Tomato;">Hello World</h1>  
<p style="color:DodgerBlue;">Lorem ipsum...</p>  
<p style="color:MediumSeaGreen;">Ut wisi enim...</p>
```

CSS Border Color

```
<h1 style="border:2px solid Tomato;">Hello World</h1>  
  
<h1 style="border:2px solid DodgerBlue;">Hello World</h1>  
  
<h1 style="border:2px solid Violet;">Hello World</h1>
```

CSS Color Values

In CSS, colors can also be specified using RGB values, HEX values, HSL values, RGBA values, and HSLA values:

Same as color name "Tomato":



`rgb(255, 99, 71)`

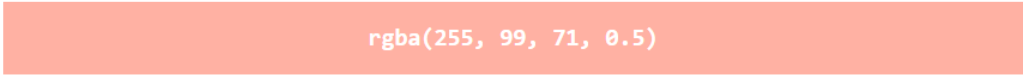


`#ff6347`

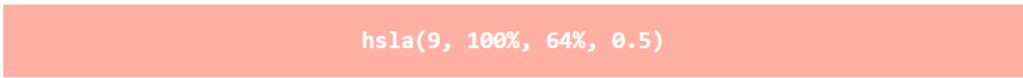


`hsl(9, 100%, 64%)`

Same as color name "Tomato", but 50% transparent:



`rgba(255, 99, 71, 0.5)`



`hsla(9, 100%, 64%, 0.5)`

Example

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p>Same as color name "Tomato":</p>
```

```
<h1 style="background-color:rgb(255, 99, 71);">rgb(255, 99, 71)</h1>
```

```
<h1 style="background-color:#ff6347;">#ff6347</h1>
```

```
<h1 style="background-color:hsl(9, 100%, 64%);">hsl(9, 100%, 64%)</h1>
```

```
<p>Same as color name "Tomato", but 50% transparent:</p>
```

```
<h1 style="background-color:rgba(255, 99, 71, 0.5);">rgba(255, 99, 71, 0.5)</h1>
```

```
<h1 style="background-color:hsla(9, 100%, 64%, 0.5);">hsla(9, 100%, 64%, 0.5)</h1>
```

```
<p>In addition to the predefined color names, colors can be specified using RGB, HEX, HSL, or even transparent colors using RGBA or HSLA color values.</p>
```

```
</body>
```

```
</html>
```


CSS Backgrounds

CSS background-color

The background-color property specifies the background color of an element.

Example

The background color of a page is set like this:

```
body {  
  background-color: lightblue;  
}
```

Other Elements

You can set the background color for any HTML elements:

```
h1 {  
  background-color: green;  
}  
  
div {  
  background-color: lightblue;  
}  
  
p {  
  background-color: yellow;  
}
```

Opacity / Transparency

The opacity property specifies the opacity/transparency of an element. It can take a value from 0.0 - 1.0. The lower value, the more transparent:

```
div {  
  background-color: green;  
  opacity: 0.3;  
}
```

CSS background-image

The background-image property specifies an image to use as the background of an element.

By default, the image is repeated so it covers the entire element.

Example

```
body {  
  background-image: url("paper.gif");  
}
```

The background image can also be set for specific elements, like the <p> element:

Example

```
p {  
  background-image: url("paper.gif");  
}
```

CSS Background Image Repeat

By default, the background-image property repeats an image both horizontally and vertically.

Some images should be repeated only horizontally or vertically, or they will look strange, like this:

Example

```
body {  
  background-image: url("gradient_bg.png");  
}
```

If the image above is repeated only horizontally (background-repeat: repeat-x;), the background will look better:

Example

```
body {  
  background-image: url("gradient_bg.png");  
  background-repeat: repeat-x;  
}
```

To repeat an image vertically, set background-repeat: repeat-y;

CSS background-repeat: no-repeat

Showing the background image only once is also specified by the background-repeat property:

Example

Show the background image only once:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
}
```

CSS background-position

The background-position property is used to specify the position of the background image.

Example

Position the background image in the top-right corner:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
}
```

The CSS Background Repeat and Position Properties

Property	Description
background-position	Sets the starting position of a background image
background-repeat	Sets how a background image will be repeated

CSS background-attachment

The background-attachment property specifies whether the background image should scroll or be fixed (will not scroll with the rest of the page):

Example

Specify that the background image should be fixed:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
  background-attachment: fixed;  
}
```

Specify that the background image should scroll with the rest of the page:

```
body {  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
  background-attachment: scroll;  
}
```

The CSS Background Attachment Property

Property	Description
background-attachment	Sets whether a background image is fixed or scrolls with the rest of the page

CSS background - Shorthand property

To shorten the code, it is also possible to specify all the background properties in one single property. This is called a shorthand property.

```
body {  
  background-color: #ffffff;  
  background-image: url("img_tree.png");  
  background-repeat: no-repeat;  
  background-position: right top;  
}
```

Example

Use the shorthand property to set the background properties in one declaration:

```
body {  
  background: #ffffff url("img_tree.png") no-repeat right top;  
}
```

CSS Borders

The CSS border properties allow you to specify the style, width, and color of an element's border.

CSS Border Style

The border-style property specifies what kind of border to display.

The following values are allowed:

- dotted - Defines a dotted border
- dashed - Defines a dashed border
- solid - Defines a solid border
- double - Defines a double border
- groove - Defines a 3D grooved border. The effect depends on the border-color value
- ridge - Defines a 3D ridged border. The effect depends on the border-color value
- inset - Defines a 3D inset border. The effect depends on the border-color value
- outset - Defines a 3D outset border. The effect depends on the border-color value
- none - Defines no border
- hidden - Defines a hidden border

The **border-style** property can have from one to four values (for the top border, right border, bottom border, and the left border).

```
p.dotted {border-style: dotted;}
p.dashed {border-style: dashed;}
p.solid {border-style: solid;}
p.double {border-style: double;}
p.groove {border-style: groove;}
p.ridge {border-style: ridge;}
p.inset {border-style: inset;}
p.outset {border-style: outset;}
p.none {border-style: none;}
p.hidden {border-style: hidden;}
p.mix {border-style: dotted dashed solid double;}
```

CSS Border Width

The border-width property specifies the width of the four borders.

The width can be set as a specific size (in px, pt, cm, em, etc) or by using one of the three pre-defined values: thin, medium, or thick.

Examples:

```
p.one {
  border-style: solid;
  border-width: 5px;
}
```

```
p.two {
  border-style: solid;
  border-width: medium;
}
```

```
p.three {
  border-style: dotted;
  border-width: 2px;
}
```

```
p.four {
  border-style: dotted;
```

```
border-width: thick;
}
```

Specific Side Widths

The border-width property can have from one to four values (for the top border, right border, bottom border, and the left border):

```
p.one {
  border-style: solid;
  border-width: 5px 20px; /* 5px top and bottom, 20px on the sides */
}
```

```
p.two {
  border-style: solid;
  border-width: 20px 5px; /* 20px top and bottom, 5px on the sides */
}
```

```
p.three {
  border-style: solid;
  border-width: 25px 10px 4px 35px; /* 25px top, 10px right, 4px bottom and 35px left */
}
```

CSS Border Color

The border-color property is used to set the color of the four borders.

```
p.one {
  border-style: solid;
  border-color: red;
}
```

```
p.two {
  border-style: solid;
  border-color: green;
}
```

```
p.three {
  border-style: dotted;
  border-color: blue;
}
```

Specific Side Colors

The border-color property can have from one to four values (for the top border, right border, bottom border, and the left border).

Example

```
p.one {  
  border-style: solid;  
  border-color: red green blue yellow; /* red top, green right, blue bottom and yellow left */  
}
```

CSS Margins

The CSS margin properties are used to create space around elements, outside of any defined borders.

With CSS, you have full control over the margins. There are properties for setting the margin for each side of an element (top, right, bottom, and left).

Margin - Individual Sides

CSS has properties for specifying the margin for each side of an element:

- margin-top
- margin-right
- margin-bottom
- margin-left

All the margin properties can have the following values:

- auto - the browser calculates the margin
- *length* - specifies a margin in px, pt, cm, etc.
- % - specifies a margin in % of the width of the containing element
- inherit - specifies that the margin should be inherited from the parent element

Tip: Negative values are allowed.

Example

Set different margins for all four sides of a <p> element:


```
p {  
  margin-top: 100px;  
  margin-bottom: 100px;  
  margin-right: 150px;  
  margin-left: 80px;  
}
```

CSS Padding

The CSS padding properties are used to generate space around an element's content, inside of any defined borders.

With CSS, you have full control over the padding. There are properties for setting the padding for each side of an element (top, right, bottom, and left).

Padding - Individual Sides

CSS has properties for specifying the padding for each side of an element:

- padding-top
- padding-right
- padding-bottom
- padding-left

All the padding properties can have the following values:

- *length* - specifies a padding in px, pt, cm, etc.
- % - specifies a padding in % of the width of the containing element
- inherit - specifies that the padding should be inherited from the parent element

Note: Negative values are not allowed.

Example

Set different padding for all four sides of a <div> element:

```
div {  
  padding-top: 50px;  
  padding-right: 30px;  
  padding-bottom: 50px;
```

```
padding-left: 80px;  
}
```

Padding - Shorthand Property

To shorten the code, it is possible to specify all the padding properties in one property.

The padding property is a shorthand property for the following individual padding properties:

- padding-top
- padding-right
- padding-bottom
- padding-left

So, here is how it works:

If the padding property has four values:

- **padding: 25px 50px 75px 100px;**
 - top padding is 25px
 - right padding is 50px
 - bottom padding is 75px
 - left padding is 100px

Example

Use the padding shorthand property with four values:

```
div {  
padding: 25px 50px 75px 100px;  
}
```

CSS height and width Values

The height and width properties may have the following values:

- auto - This is default. The browser calculates the height and width
- length - Defines the height/width in px, cm, etc.
- % - Defines the height/width in percent of the containing block

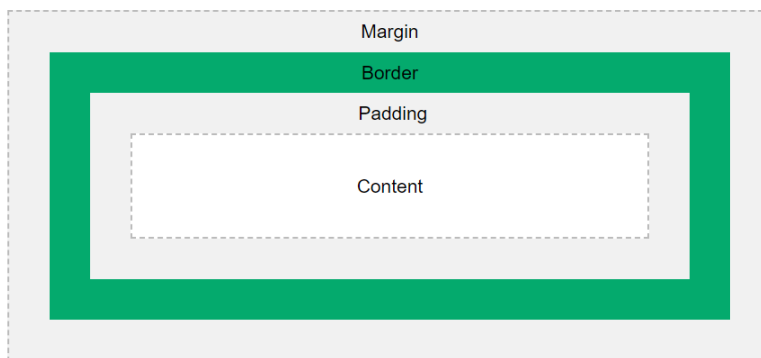
- initial - Sets the height/width to its default value
- inherit - The height/width will be inherited from its parent value
- Set the height and width of a <div> element:

```
div {  
    height: 200px;  
    width: 50%;  
    background-color: powderblue;  
}
```

The CSS Box Model

In CSS, the term "box model" is used when talking about design and layout.

The CSS box model is essentially a box that wraps around every HTML element. It consists of: content, padding, borders and margins. The image below illustrates the box model:



Explanation of the different parts:

- **Content** - The content of the box, where text and images appear
- **Padding** - Clears an area around the content. The padding is transparent
- **Border** - A border that goes around the padding and content
- **Margin** - Clears an area outside the border. The margin is transparent

Example

Demonstration of the box model:

```
div {  
  width: 300px;  
  border: 15px solid green;  
  padding: 50px;  
  margin: 20px;  
}
```

Width and Height of an Element

```
div {  
  width: 320px;  
  height: 50px;  
  padding: 10px;  
  border: 5px solid gray;  
  margin: 0;  
}
```

CSS Text**Text Color**

The color property is used to set the color of the text. The color is specified by:

- a color name - like "red"
- a HEX value - like "#ff0000"
- an RGB value - like "rgb(255,0,0)"

```
body {  
  color: blue;  
}
```

```
h1 {  
  color: green;  
}
```

Text Color and Background Color

```
body {  
  background-color: lightgrey;  
  color: blue;  
}
```

```
h1 {  
  background-color: black;  
  color: white;  
}
```

```
div {  
  background-color: blue;  
  color: white;  
}
```

Text Alignment

The text-align property is used to set the horizontal alignment of a text.

A text can be left or right aligned, centered, or justified.

The following example shows center aligned, and left and right aligned text (left alignment is default if text direction is left-to-right, and right alignment is default if text direction is right-to-left):

```
h1 {  
  text-align: center;  
}  
h2 {  
  text-align: left;  
}  
h3 {  
  text-align: right;  
}
```

When the text-align property is set to "justify", each line is stretched so that every line has equal width, and the left and right margins are straight (like in magazines and newspapers):

Example

```
div {  
  text-align: justify;  
}
```

What is JavaScript

JavaScript is a programming language that executes on the browser. It turns static HTML web pages into interactive web pages by dynamically updating content, validating form data, controlling multimedia, animate images, and almost everything else on the web pages. JavaScript is the third most important web technology after HTML and CSS. JavaScript can be used to create web and mobile applications, build web servers, create games, etc.

JavaScript Example

JavaScript can be used in various activities like data validation, displaying popup messages, handling events of HTML elements, modifying CSS, etc. The following sample form uses JavaScript to validate data and change the color of the form.



First Name: *

Middle Name:

Last Name: *

Date of Birth: 

Address:

City: *

Zip Code: *

The responsive UI and menu of this website is also using JavaScript. There is no website in this world that does not use JavaScript or JavaScript-based frameworks.

JavaScript History

In early 1995, Brendan Eich from Netscape designed and implemented a new language for non-java programmers to give newly added Java support in Netscape navigator. It was initially named Mocha, then LiveScript, and finally JavaScript.

Nowadays, JavaScript can execute not only on browsers but also on the server or any device with a JavaScript Engine. For example, Node.js is a framework based on JavaScript that executes on the server.

JavaScript Engine

JavaScript engine interprets, compiles, and executes JavaScript code. It also does memory management, JIT compilation, type system, etc. Different browsers use different JavaScript engines, as listed in the below table.

Browser	JavaScript Engine
Edge	Chromium with Blink and V8 engines
Chrome	V8
FireFox	SpiderMonkey
Safari	Nitro

Setup JavaScript Development Environment

we must have at least two things, a browser, and an editor to write the JavaScript code.

Browser

Mostly, you will have a browser already installed on your PC, Microsoft Edge on the Windows platform, and Safari on Mac OS.

You can also install the following browser as per your preference:

- Microsoft Edge
- Google Chrome
- Mozilla FireFox
- Safari
- Opera

IDEs for JavaScript Application Development

You can write JavaScript code using a simple editor like Notepad. However, you can install any open-sourced or licensed IDE (Integrated Development Environment) to get the advantage of IntelliSense support for JavaScript and syntax error/warning highlighter for rapid development.

The followings are some of the well-known JavaScript editors:

- Visual Studio Code (Free, cross-platform)
- Eclipse (Free, cross-platform)
- Atom (Free, cross-platform)
- Notepad++ (Free, Windows)
- Code Lobster (Free, cross-platform)
- WebStorm (Paid, cross-platform)

Online Editors for JavaScript

Use the online editor to quickly execute the JavaScript code without any installation. The followings are free online editors:

- jsfiddle.net
- jsbin.com
- playcode.io

HTML script Tag

The HTML script tag `<script>` is used to embed data or executable client side scripting language in an HTML page. Mostly, JavaScript or JavaScript based API code inside a `<script></script>` tag.

The following is an example of an HTML page that contains the JavaScript code in a `<script>` tag.

Example: JavaScript in a `<script>` Tag

```
<!DOCTYPE html>
<html>
<head>
</head>
<body>
  <h1> JavaScript Tutorials</h1>

  <script>
    //write JavaScript code here..
    alert('Hello, how are you?')
```

```
</script>
</body>
</html>
```

In the above example, a `<script></script>` tag contains the JavaScript `alert('Hello, how are you?')` that display a message box.

HTML v4 requires the `type` attribute to identify the language of script code embedded within script tag. This is specified as MIME type e.g. `'text/javascript'`, `'text/ecmascript'`, `'text/vbscript'`, etc.

HTML v5 page does not require the `type` attribute because the default script language is `'text/javascript'` in a `<script>` tag.

An HTML page can contain multiple `<script>` tags in the `<head>` or `<body>` tag. The browser executes all the script tags, starting from the first script tag from the beginning.

Scripts without `async`, `defer` or `type="module"` attributes, as well as inline scripts, are fetched and executed immediately, before the browser continues to parse the page. Consider the following page with multiple script tags.

Example: Multiple `<script>` Tags

```
<!DOCTYPE html>
<html>
<head>
  <script>
    alert('Executing JavaScript 1')
  </script>
</head>
<body>
  <h1> JavaScript Tutorials</h1>

  <script>
    alert('Executing JavaScript 2')
  </script>

  <p>This page contains multiple script tags.</p>

  <script>
    alert('Executing JavaScript 3')
  </script>
</body>
</html>
```

Above, the first `<script>` tag containing `alert('Executing JavaScript 1')` will be executed first, then `alert('Executing JavaScript 2')` will be executed, and then `alert('Executing JavaScript 3')` will be executed.

The browser loads all the scripts included in the `<head>` tag before loading and rendering the `<body>` tag elements. So, always include JavaScript files/code in the `<head>` that are going to be used while rendering the UI.

Reference the External Script File

A `<script>` tag can also be used to include an external script file to an HTML web page by using the `src` attribute.

If you don't want to write inline JavaScript code in the `<script></script>` tag, then you can also write JavaScript code in a separate file with `.js` extension and include it in a web page using `<script>` tag and reference the file via `src` attribute.

Example: JavaScript in a `<script>` Tag

```
<!DOCTYPE html>
<html>
<head>
  <script src="/MyJavaScriptFile.js" ></script>
</head>
<body>
  <h1> JavaScript for SYCS</h1>

</body>
</html>
```

Above, the `<script src="/MyJavaScriptFile.js">` points to the external JavaScript file using the `src="/MyJavaScriptFile.js"` attribute where the value of the `src` attribute is the path or url from which a file needs to be loaded in the browser.

Global Attributes

The `<script>` can contain the following global attributes:

Attribute	Usage
async	<code><script async></code> executes the script asynchronously along with the rest of the page.
crossorigin	<code><script crossorigin="anonymous use-credentials"></code> allows error logging for sites which use a separate domain for static media. Value <code>anonymous</code> do not send credentials, whereas <code>use-credentials</code> sends the credentials.
defer	<code><script defer></code> executes the script after the document is parsed and before firing <code>DOMContentLoaded</code> event.
src	<code><script src="uri\path to resource"></code> specifies the URI/path of an external script file;
type	<code><script type="text\javascript"></code> specifies the type of the containing script e.g. <code>text\javascript</code> , <code>text\html</code> , <code>text\plain</code> , <code>application\json</code> , <code>application\pdf</code> , etc.
referrerpolicy	<code><script referrerpolicy="no-referrer"></code> specifies which referrer information to send when fetching a script. Values can be <code>no-referrer</code> , <code>no-referrer-when-downgrade</code> , <code>origin</code> , <code>same-origin</code> , <code>strict-origin</code> , etc.
integrity	<code><script integrity="sha384-oqVuAfXRKap7fdgc"></code> specifies that a user agent can use to verify that a fetched resource has been delivered free of unexpected manipulation.
nomodule	<code><script nomodule></code> specifies that the script should not be executed in browsers supporting ES2015 modules.

JavaScript Syntax

```
<script>
  //Write javascript code here...

</script>
```

Character Set

JavaScript uses the unicode character set, so allows almost all characters, punctuations, and symbols.

Case Sensitive

JavaScript is a case-sensitive scripting language. So, name of functions, variables and keywords are case sensitive. For example, myfunction and MyFunction are different, Name is not equal to nAme, etc.

Variables

```
<script>
  var name = "Steve";
  id = 10;
</script>
```

Semicolon

JavaScript statements are separated by a semicolon. However, it is not mandatory to end a statement with a semicolon, but it is recommended.

Example: JavaScript Statements

```
<script>
  var one = 1; two = 2; three = 3; //three different statements

  var four = 4; //single statement
  var five = "Five" //single statement without ;
</script>
```

Whitespaces

JavaScript ignores multiple spaces and tabs. The following statements are the same.

Example: Whitespaces in JavaScript

```
<script>
  var one =1;
  var one  = 1;
  var one   =    1;
</script>
```

Code Comments

A comment is single or multiple lines, which give some information about the current program. Comments are not for execution.

Write comment after double slashes // or write multiple lines of comments between /* and */

Example: Comment JavaScript Code

```
<script>
```

```
var one =1; // this is a single line comment
```

```
/* this  
is multi line  
comment*/
```

```
var two = 2;  
var three = 3;  
</script>
```

String

A string is a text in JavaScript. The text content must be enclosed in double or single quotation marks.

Example: String in JavaScript

```
<script>  
var msg = "Hello World" //JavaScript string in double quotes
```

```
var msg = 'Hello World' //JavaScript string in single quotes  
</script>
```

Number

JavaScript allows you to work with any type of number like integer, float, hexadecimal etc.

Number must **NOT** be wrapped in quotation marks.

Example: Numbers in JavaScript

```
<script>  
var num = 100;
```

```
var flot = 10.5;  
</script>
```

Boolean

As in other languages, JavaScript also includes true and false as a boolean value.

Example: Booleans in JavaScript

```
<script>  
var yes = true;
```

```
var no = false;  
</script>
```

Keywords

Keywords are reserved words in JavaScript, which cannot be used as variable names or function names.

The following table lists some of the keywords used in JavaScript.

JavaScript Reserved Keywords		
var	function	if
else	do	while
for	switch	break
continue	return	try
catch	finally	debugger
case	class	this
default	false	true
in	instanceOf	typeof
new	null	throw
void	width	delete

JavaScript Message Boxes: alert(), confirm(), prompt()

JavaScript provides built-in global functions to display popup message boxes for different purposes.

- **alert(message):** Display a popup box with the specified message with the OK button.
- **confirm(message):** Display a popup box with the specified message with OK and Cancel buttons.
- **prompt(message, defaultValue):** Display a popup box to take the user's input with the OK and Cancel buttons.

Note:

In JavaScript, global functions can be accessed using the window object like `window.alert()`, `window.confirm()`, `window.prompt()`.

alert()

The `alert()` function displays a message to the user to display some information to users. This alert box will have the **OK** button to close the alert box.

Syntax:

```
window.alert([message]);
```

The `alert()` function takes a parameter of any type e.g., string, number, boolean etc. So, no need to convert a non-string type to a string type.

Example:

```

<!DOCTYPE html>
<html>
<body>
    <h1>Demo: alert()</h1>
    <button onclick="displayInfo()">Display Messages</button>

    <script>
        function displayInfo(){
            alert("This is an alert message box."); // display string message

            alert('This is a numer: ' + 100); // display result of a concatenation

            alert(100); // display number

            alert(Date()); // display current date
        }
    </script>
</body>
</html>

```

confirm()

Use the confirm() function to take the user's confirmation before starting some task. For example, you want to take the user's confirmation before saving, updating or deleting data.

Syntax:

```
bool window.confirm([message]);
```

The confirm() function displays a popup message to the user with two buttons, OK and Cancel.

The confirm() function returns true if a user has clicked on the OK button or returns false if clicked on the Cancel button. You can use the return value to process further.

```

<!DOCTYPE html>
<html>
<body>
    <h1>Demo: confirm() </h1>
    <button onclick="save()">Save Data</button>
    <p id="msg"></p>

    <script>
        function save(){
            var userPreference;

            if (confirm("Do you want to save changes?") == true) {
                userPreference = "Data saved successfully!";
            } else {
                userPreference = "Save Canceled!";
            }

            document.getElementById("msg").innerHTML = userPreference;
        }
    </script>

```

```
</script>
</body>
</html>
```

prompt()

Use the prompt() function to take the user's input to do further actions. For example, use the prompt() function in the scenario where you want to calculate EMI based on the user's preferred loan tenure.

Syntax:

```
string prompt([message], [defaultValue]);
```

The prompt() function takes two parameters. The first parameter is the message to be displayed, and the second parameter is the default value in an input box.

```
<!DOCTYPE html>
<html>
<body>
    <h1>Demo: prompt()</h1>
    <button onclick="myinput()">Click to enter your name</button>
    <p id="msg"></p>

    <script>
        function myinput(){
            var name = prompt("Enter Your Name:");

            if (name == null || name == "") {
                document.getElementById("msg").innerHTML = "You did not
entert anything. Please enter your name again";
            }
            else
            {
                document.getElementById("msg").innerHTML = "You enterted: " +
name;
            }
        }
    </script>
</body>
</html>
```

JavaScript Values

In JavaScript, you can have two types of values.

- Fixed values (Literals)
- Variables (Dynamic values)

```
<html>
<body>
  <script>
    document.write(10); // Number Literal
    document.write("<br />"); // To add line-break
    document.write("Hello"); // String Literal
  </script>
</body>
</html>
```

JavaScript Variables

In JavaScript, variables are used to store the dynamic data.

You can use the below keyword to define variables in JavaScript.

- var
- let
- const

Example of let :

```
<html>
<body>
  <script>
    let a = 5; // Variable Declaration
    document.write(a); // Using variable
    document.write("<br>");
    let b = "One";
    document.write(b);
  </script>
</body>
</html>
```

Example of var :

```
<html>
<body>
  <script>
    function getSum(first, second) {
      var sum = first * second;
      document.write("The product of " + first + " and " + second + " is " + sum);
    }
    let first = 3;
    const second = 4;
    getSum(first, second);
  </script>
</body>
</html>
```

Example of const :

```
<html>
<body>
  <script>
    const x = 3; y = 5;
    var z = x + y ;
    document.write(z + "<br>");
    var z = '10' + 3
    document.write(z + "<br>");
    var z = '10' + '3';
    document.write(z + "<br>");
  </script>
</body>
</html>
```

Primitive Data Types in JavaScript

Primitive data types in javascript refer to those data types that are defined on the most basic level of the language. These Javascript data types cannot be deconstructed further, nor do they have any pre-defined properties or methods. They are only used to store items of *their type*.

There are 7 primitive data types in JavaScript:

1. **Numbers**
2. **BigInt**
3. **String**
4. **Boolean**
5. **Undefined**
6. **Null**
7. **Symbol**

Following is the table for all the primitive data types in javascript with their descriptions and examples.

Data Type	Description	Example
undefined	Represents an undefined value.	let x; // x is undefined
null	Represents an null value.	let y = null; // y is null
boolean	Represents a boolean value, which can be true or false	let a = true; // a is true
number	Represents a numeric value, either integer or floating point.	let b = 5; // b is 5
string	Represents a sequence of characters, enclosed in quotes.	let c = "hello"; // c is "hello"
symbol	Represents a unique identifier, used primarily for object properties.	let d = Symbol("description"); // d is a unique symbol
BigInt	Represents integers with arbitrary precision.	let e = 9007199254740992n; // e is a BigInt

JavaScript If-else

The **JavaScript if-else statement** is used *to execute the code whether condition is true or false*. There are three forms of if statement in JavaScript.

1. If Statement
2. If else statement
3. if else if statement
- 4.

JavaScript If statement

It evaluates the content only if expression is true. The signature of JavaScript if statement is given below.

1. if(expression){
2. //content to be evaluated
3. }

Flowchart of JavaScript If statement

Let's see the simple example of if statement in javascript.

```
<script>
var a=20;
if(a>10){
document.write("value of a is greater than 10");
}
</script>
```

JavaScript If...else Statement

It evaluates the content whether condition is true or false. The syntax of JavaScript if-else statement is given below.

```
if(expression){
//content to be evaluated if condition is true
}
else{
//content to be evaluated if condition is false
}
<script>
var a=20;
if(a%2==0){
document.write("a is even number");
}
else{
document.write("a is odd number");
}
</script>
```

JavaScript If...else if statement

It evaluates the content only if expression is true from several expressions. The signature of JavaScript if else if statement is given below.

```
if(expression1){
//content to be evaluated if expression1 is true
}
```

```

else if(expression2){
//content to be evaluated if expression2 is true
}
else if(expression3){
//content to be evaluated if expression3 is true
}
else{
//content to be evaluated if no expression is true
}

```

Let's see the simple example of if else if statement in javascript.

```

<script>
var a=20;
if(a==10){
document.write("a is equal to 10");
}
else if(a==15){
document.write("a is equal to 15");
}
else if(a==20){
document.write("a is equal to 20");
}
else{
document.write("a is not equal to 10, 15 or 20");
}
</script>

```

JavaScript Switch

The **JavaScript switch statement** is used *to execute one code from multiple expressions*. It is just like else if statement that we have learned in previous page. But it is convenient than *if..else..if* because it can be used with numbers, characters etc.

The signature of JavaScript switch statement is given below.

```

switch(expression){
case value1:
code to be executed;
break;
case value2:
code to be executed;
break;
.....

default:
code to be executed if above values are not matched;
}

```

Let's see the simple example of switch statement in javascript.

```

<script>
var grade='B';

```

```
var result;
switch(grade){
case 'A':
result="A Grade";
break;
case 'B':
result="B Grade";
break;
case 'C':
result="C Grade";
break;
default:
result="No Grade";
}
document.write(result);
</script>
```

JavaScript Loops

The **JavaScript loops** are used *to iterate the piece of code* using for, while, do while or for-in loops. It makes the code compact. It is mostly used in array.

There are four types of loops in JavaScript.

1. for loop
2. while loop
3. do-while loop
4. for-in loop

1) JavaScript For loop

The **JavaScript for loop** *iterates the elements for the fixed number of times*. It should be used if number of iteration is known. The syntax of for loop is given below.

1. for (initialization; condition; increment)
2. {
3. code to be executed
4. }

Let's see the simple example of for loop in javascript.

```
<script>
for (i=1; i<=5; i++)
{
document.write(i + "<br/>")
}
</script>
```

JavaScript while loop

The **JavaScript while loop** *iterates the elements for the infinite number of times*. It should be used if number of iteration is not known. The syntax of while loop is given below.

```
while (condition)
{
code to be executed
}
```

```
}
```

Let's see the simple example of while loop in javascript.

```
<script>
var i=11;
while (i<=15)
{
document.write(i + "<br/>");
i++;
}
</script>
```

JavaScript do while loop

The **JavaScript do while loop** *iterates the elements for the infinite number of times* like while loop. But, code is *executed at least* once whether condition is true or false. The syntax of do while loop is given below.

```
do{
    code to be executed
}while (condition);
<script>
var i=21;
do{
document.write(i + "<br/>");
i++;
}while (i<=25);
</script>
```

JavaScript Function Syntax

The syntax of declaring function is given below.

```
function functionName([arg1, arg2, ...argN]){
    //code to be executed
}
```

JavaScript Function Example

Let's see the simple example of function in JavaScript that does not has arguments.

```
<script>
function msg(){
alert("hello! this is message");
}
</script>
<input type="button" onclick="msg()" value="call function"/>
```

JavaScript Function Arguments

We can call function by passing arguments. Let's see the example of function that has one argument.

```
<script>
function getcube(number){
alert(number*number*number);
```

```

}
</script>
<form>
<input type="button" value="click" onclick="getcube(4)"/>
</form>

```

Function with Return Value

We can call function that returns a value and use it in our program. Let's see the example of function that returns value.

```

<script>
function getInfo(){
return "hello SYCS! How r u?";
}
</script>
<script>
document.write(getInfo());
</script>

```

JavaScript String

The **JavaScript string** is an object that represents a sequence of characters.

There are 2 ways to create string in JavaScript

1. By string literal
2. By string object (using new keyword)

By string literal

The string literal is created using double quotes. The syntax of creating string using string literal is given below:

1. `var stringname="string value";`

Let's see the simple example of creating string literal.

```

<script>
var str="This is string literal";
document.write(str);
</script>

```

By string object (using new keyword)

The syntax of creating string object using new keyword is given below:

```
var stringname=new String("string literal");
```

Here, **new keyword** is used to create instance of string.

Let's see the example of creating string in JavaScript by new keyword.

```

<script>
var stringname=new String("hello this is SYCS string");
document.write(stringname);
</script>

```

JavaScript String Methods

JavaScript String charAt(index) Method

The JavaScript String charAt() method returns the character at the given index.

```

<script>
var str="java String";

```

```
document.write(str.charAt(2));
```

```
</script>
```

JavaScript String concat(str) Method

The JavaScript String concat(str) method concatenates or joins two strings.

```
<script>
```

```
var s1="javascript ";
```

```
var s2="concat example";
```

```
var s3=s1.concat(s2);
```

```
document.write(s3);
```

```
</script>
```

JavaScript String indexOf(str) Method

The JavaScript String indexOf(str) method returns the index position of the given string.

```
<script>
```

```
var s1="javascript from string indexof";
```

```
var n=s1.indexOf("from");
```

```
document.write(n);
```

```
</script>
```

JavaScript String toLowerCase() Method

The JavaScript String toLowerCase() method returns the given string in lowercase letters.

```
<script>
```

```
var s1="JavaScript toLowerCase Example";
```

```
var s2=s1.toLowerCase();
```

```
document.write(s2);
```

```
</script>
```

JavaScript Objects

A JavaScript object is an entity having state and behavior (properties and method). For example: car, pen, bike, chair, glass, keyboard, monitor etc.

JavaScript is an object-based language. Everything is an object in JavaScript.

JavaScript is template based not class based. Here, we don't create class to get the object. But, we directly create objects.

Creating Objects in JavaScript

There are 3 ways to create objects.

1) JavaScript Object by object literal

The syntax of creating object using object literal is given below:

1. object={property1:value1,property2:value2.....propertyN:valueN}

As you can see, property and value is separated by : (colon).

Let's see the simple example of creating object in JavaScript.

1. **<script>**

2. emp={id:102,name:"Shyam Kumar",salary:40000}

3. document.write(emp.id+" "+emp.name+" "+emp.salary);

4. **</script>**

Output of the above example

```
102 Shyam Kumar 40000
```

2) By creating instance of Object

The syntax of creating object directly is given below:

1. `var objectname=new Object();`

Here, **new keyword** is used to create object.

Let's see the example of creating object directly.

```
<script>
var emp=new Object();
emp.id=101;
emp.name="Ravi Malik";
emp.salary=50000;
document.write(emp.id+" "+emp.name+" "+emp.salary);
</script>
```

Output of the above example

101 Ravi 50000

3) By using an Object constructor

Here, you need to create function with arguments. Each argument value can be assigned in the current object by using this keyword.

The **this keyword** refers to the current object.

The example of creating object by object constructor is given below.

```
<script>
function emp(id,name,salary){
this.id=id;
this.name=name;
this.salary=salary;
}
e=new emp(103,"Vimal Jaiswal",30000);

document.write(e.id+" "+e.name+" "+e.salary);
</script>
```

Output of the above example

103 Vimal Jaiswal 30000

Defining method in JavaScript Object

We can define method in JavaScript object. But before defining method, we need to add property in the function with same name as method.

The example of defining method in object is given below.

```
<script>
function emp(id,name,salary){
this.id=id;
this.name=name;
this.salary=salary;

this.changeSalary=changeSalary;
function changeSalary(otherSalary){
this.salary=otherSalary;
```

```

    }
    }
    e=new emp(103,"Sonoo Jaiswal",30000);
    document.write(e.id+" "+e.name+" "+e.salary);
    e.changeSalary(45000);
    document.write("<br>" +e.id+" "+e.name+" "+e.salary);
</script>

```

Output of the above example
 103 Sonoo Jaiswal 30000
 103 Sonoo Jaiswal 45000
 JavaScript Object Methods

The Window History Object

The **history object** contains the URLs visited by the user (in the browser window).
 The **history object** is a property of the **window object**.
 The **history object** is accessed with:

window.history or just history:

Examples

```

let length = window.history.length;
let length = history.length;

```

History Object Properties and Methods

Property/Method	Description
back()	Loads the previous URL (page) in the history list
forward()	Loads the next URL (page) in the history list
go()	Loads a specific URL (page) from the history list
length	Returns the number of URLs (pages) in the history list

Window History

The history object contains the URLs visited by the user (in the browser window).
 The history object is a property of the window object.
 The history object is accessed with:
 window.history or just history:

Examples

```

let length = window.history.length;
let length = history.length;
History Object Properties and Methods

```

Property/Method	Description
-----------------	-------------

back()	Loads the previous URL (page) in the history list
forward()	Loads the next URL (page) in the history list
go()	Loads a specific URL (page) from the history list
length	Returns the number of URLs (pages) in the history list

Window history.back()

Example

Create a back button on a page:

```
<button onclick="history.back()">Go Back</button>
```

Click on Go Back to see how it works.

(Will only work if a previous page exists in your history list)

Description

The history.back() method loads the previous URL (page) in the history list.

The history.back() method only works if a previous page exists.

Window history.forward()

Example

Create a forward button on a page:

```
<button onclick="history.forward()">Go Forward</button>
```

Window history.go()

Create a button to go back two pages:

```
<button onclick="history.go(-2)">Go Back 2 Pages</button>
```

Windows history.length

Example

Get the number of URLs in the history list:

```
let length = history.length;
```

Window Navigator

The Window Navigator Object

The **navigator object** contains information about the browser.

The **navigator object** is a property of the **window object**.

The **navigator object** is accessed with:

window.navigator or just navigator:

Examples

```
let url = window.navigator.language;
```

OR

```
let url = navigator.language;
```

Navigator Object Properties

Property	Description
appCodeName	Returns browser code name
appName	Returns browser name
appVersion	Returns browser version
cookieEnabled	Returns true if browser cookies are enabled
geolocation	Returns a geolocation object for the user's location
language	Returns browser language
onLine	Returns true if the browser is online
platform	Returns browser platform
product	Returns browser engine name
userAgent	Returns browser user-agent header

Navigator Object Methods

Method	Description
javaEnabled()	Returns true if the browser has Java enabled
taintEnabled()	Removed in JavaScript version 1.2 (1999).

Window Location

The Window Location Object

The **location object** contains information about the current URL.

The **location object** is a property of the **window object**.

The **location object** is accessed with:

window.location or just location

Examples

```
let origin = window.location.origin;
```

OR

```
let origin = location.origin;
```

Location Object Properties

Property	Description
hash	Sets or returns the anchor part (#) of a URL
host	Sets or returns the hostname and port number of a URL
hostname	Sets or returns the hostname of a URL
href	Sets or returns the entire URL

origin	Returns the protocol, hostname and port number of a URL
pathname	Sets or returns the path name of a URL
port	Sets or returns the port number of a URL
protocol	Sets or returns the protocol of a URL
search	Sets or returns the querystring part of a URL

Location Object Methods

Method	Description
assign()	Loads a new document
reload()	Reloads the current document
replace()	Replaces the current document with a new one

What is XML?

XML stands for e**X**tensible **M**arkup **L**anguage. It is a language (not a programming language) that uses the markup and can extend. It is derived from **S**tandard **G**eneralized **M**arkup **L**anguage(SGML). XML also uses DTDs (**D**ocument **T**ype **D**efinitions) to define the XML document structure.

XML is not for handling computational operations and algorithms. Thus, XML is not a programming language. The main goal is to transport data not to display information. XML bridges the gap between human readability and machine readability. Unlike HTML tags, XML tags are self-descriptive.

XML is an open format. The filename extension of XML is .xml

History of XML

XML started way back in 1996 and was first published in 1998. **World Wide Web Consortium (W3C)** is the developer of **XML**, and it became a **W3C recommendation** in 1998.

There are two versions of XML.

1. XML 1.0
2. XML 1.1

XML 1.1 is the latest version. Yet, **XML 1.0** is the most used version.

Editors of XML are:

- Tim Bray,
- Jean Paoli,
- C. M. Sperberg,
- Eve Maler,
- François Yergeau.

XML Features

Here are some important features of XML:

- It is extensible and human-readable.
- It is platform and language independent.
- It preserves white space.
- Overall simplicity.
- Self-descriptive nature.
- It separates data from HTML.
- XML tags are not predefined. You need to define your customized tags.
- XML was designed to carry data, not to display that data.
- Mark-up code of XML is easy to understand for a human.
- Well-structured format is easy to read and write from programs.
- XML is an extensible markup language like HTML.

XML Encoding

Encoding is the conversion of Unicode characters to their binary representation. UTF is used for XML encoding. **UTF** stands for **UCS (UCS stands for Universal Character Set) Transformation Format**.

Mainly, there are two types of UTF encoding.

1. UTF-8 : UTF-8 uses 8-bits to represent the characters.

Example:

```
<?xml version="1.0" encoding="UTF-8"?>
```

2. UTF-16

It uses 16-bits to represent the characters.

Example:

```
<?xml version="1.0" encoding="UTF-16"?>
```

You can use encoding inside the XML declaration. UTF-8 is the default encoding in XML.

XML Syntax

The below code segment shows the basic XML syntax.

```
<?xml version = "1.0" encoding = "UTF-8" ?>
<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>
```

XML Declaration

XML declaration consists of the XML version, character encoding or/and standalone status. The declaration is optional.

Syntax for XML Declaration

The below code segment shows the syntax for XML declaration.

```
<?xml version="version_number," encoding="character_encoding" standalone="yes_or_no" ?>
```

XML Declaration Rules

Following are XML declaration rules.

- If the XML declaration is present, it must be the first thing that appears.
- The XML declaration is case sensitive, and it must start with the lowercased **<?xml**.
- It has no closing tag.

Example of XML Declaration

Following code segment shows an example of an XML declaration.

```
<?xml version="1.0" encoding="UTF-8" standalone="no" ?>
```

XML Comments

Comments are optional. Adding comments help to understand the document content.

Syntax for XML Comments

A comment begins with **<!--** and ends with **-->**.

Following code segment shows the syntax for XML comments.

```
<!-- Add your comment here -->
```

XML Tags and Elements

Tags work as pairs except for declarations. Every tag pair consists of an **opening tag** (also known as the **start tag**) and a **closing tag** (also known as the **end tag**).

Tag names are enclosed in <>. For a particular tag pair, the start and end tags must be identical except the end tag has / after the <.

<name>...</name>

Anything between the opening and closing tags is referred to as **content**.

Opening tag, content, and closing tag, altogether, is referred to as an **element**.

Opening tag + content + closing tag = an element

Note: Elements may also contain attributes. You will learn the attributes very soon.

Let us consider the below element.

<age>20</age>

In the above element,

- **age** is the **name of the element**.

Note: Tag name also referred to as an **element** or **element name**.

- **<age>** – opening tag
- **25** – content
- **</age>** – closing tag.

If there is no content between the tags, as shown below, it referred to as **empty tags**.

<result></result>

XML Tag and Element Rules

Following list shows XML tag and element rules.

- Tags are case sensitive.

Example:

Correct:

<age>20</age>

Wrong:

<age>20</Age>

Note: AGE, Age, and age are three different names in XML.

- All XML documents must contain a single root element.
- All elements must have a closing tag (except for declarations).
- A tag name must begin with a letter or an underscore, and it cannot start with the **XML**.
- A tag name can contain letters, digits, hyphens, underscores, and periods. Hyphens underscore, and periods are the only punctuation marks allowed.
- A tag name cannot contain spaces.
- All elements must be nested properly.

Example:

Correct:

<u>This text is bold and italic</u>

Wrong:

<u>This text is bold and italic.</u>

XML Attributes

Attribute for an element is placed after the tag name in the start tag. You can add more than one attribute for a single element with different attribute names.

Let's consider the below XML document.

```
<company name="ABC Holdings" location="London">
  <chairman>Mr. John</chairman>
  <gm>Mr. Wood</gm>
</company>
```

There are two attributes in the **company** element, i.e. **name** and **location**.

Let's study the **name** attribute,

- **name="ABC Holdings"** – an attribute
- **name** – attribute name
- **ABC Holdings** – attribute value

Note: An **attribute name** is also known as an **attribute**.

Also, note that in the above example, the **company** is the **root** element.

XML Attribute Rules

The below list shows XML attribute rules.

- Attribute values must be within quotes.
- An element cannot contain several attributes with the same name.

Attribute versus Element

Are you still confused about the difference between an attribute and an element? Here is another example.

Let's consider documents A and B given below.

Document A:

```
<teacher subject="English">
  <name>Mr. John</name>
  <qualification>Graduate</qualification>
</teacher>
```

Document B:

```
<teacher>
  <subject>English</subject>
  <name>Mr. John</name>
  <qualification>Graduate</qualification>
</teacher>
```

In document A, the **subject** is an **attribute**.

In document B, the **subject** is an **element**.

XML Entities

What are XML Entities?

In simple terms, entities are a way of representing special characters. Entities are also known as **entity references**.

Why You Need XML Entities?

Some characters (such as **"**, **&**, **<**, and so on) are reserved in XML. They are referred to as **special characters** and cannot be directly used for other purposes.

For example, the **<** and **>** symbols are used for **tags**. You cannot directly type from the keyboard for **less than** and **greater than** signs. Instead, you need to use entities.

Following table shows some of the popular XML entities.

Character	Description	Entity Name	Usage
"	Quotation mark (double quote)	quot	"
&	Ampersand	amp	&
'	Apostrophe (single quote)	apos	'
<	Less than sign	lt	<
>	Greater than sign	gt	>

Example:

```
<friend>
```

```
    <name>My friends are Alice &amp; Jane.</name>
```

```
</friend>
```

HTML versus XML

Similarities between HTML and XML

Following list shows the similarities between HTML and XML.

- Both are open formats.
- Both are markup languages.
- Both use tags and attributes to describe the content.

Differences between HTML and XML

Even though XML is like HTML, XML is not a replacement for HTML. There are some significant differences between HTML and XML as well.

Following list table show a comparison between HTML and XML.

	HTML	XML
Stands for	Hypertext Markup Language	Extensible Markup Language
Type of language	A predefined markup language.	A framework for specifying markup languages.
Structural details	Not provided.	Provided.
Purpose	Used to display data.	Used to transport data
Driven by	Format driven.	Content-driven.
Nature	Has a static nature.	Has a dynamic nature.
Tag type	Predefined tags.	User-defined tags.
Tag limit	A limited number of tags are available.	Tags are extensible.
Closing tags	It is not necessary to use closing tags (but recommended to use closing tags).	Closing tags are mandatory.
Namespace support	Not supported.	Supported.
Case sensitivity	Tags are not case-sensitive.	Tags are case-sensitive.
White space	White space cannot preserve (can ignore white space).	White space preserved (cannot ignore white space).

Parsing in JavaScript	Not needed any extra application.	Need DOM implementation.
Code nesting	Not necessarily needed.	Needed.
Errors	Can ignore small errors.	Errors are not allowed.
Filename Extension	.html or .htm	.xml
Size	Comparatively large.	Comparatively small.
Quotes	Quotes are not required for attribute values.	Required for XML attribute values.
Object support	Offers native object support.	Objects have to be expressed by conventions.
Null support	Natively recognizes the null value.	Need to use xsi:nil on elements.
Formatting decisions	Provides direct mapping for application data.	Require more significant effort.
Learning curve	Less steep learning curve compared to XML.	Steep learning curve.

Basic HTML Syntax

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>

```

Basic XML Syntax

```

<?xml version = "1.0" encoding = "UTF-8" ?>
<root>
  <child>
    <subchild>.....</subchild>
  </child>
</root>

```

Same example with HTML and XML

With HTML

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta http-equiv="X-UA-Compatible" content="IE=edge">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Document</title>
</head>
<body>
  <p>Book</p>
  <p>Name: Anna Karenina</p>
  <p>Author: Leo Tolstoy</p>
  <p>Publisher: The Russian Messenger</p>
</body>
</html>

```

With XML

```

<?xml version = "1.0" encoding = "UTF-8" ?>
<book>
  <name>Anna Karenina</name>
  <author>Leo Tolstoy</author>
  <publisher>The Russian Messenger</publisher>
</book>

```

XML DTD

What is DTD?

DTD stands for **Document Type Definition**. It defines the structure of an XML document using some legal elements. XML DTD is optional.

DTD Rules

Following list shows DTD rules.

- If DTD is present, it must appear at the start of the document (only the XML declaration can appear above the DTD).
- The element declaration must start with an ! mark.
- The DTD name and element type of the root element must be the same.

Examples of DTD

Example of an internal DTD:

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE student [
  <!ELEMENT student (firstname,lastname,school)>
  <!ELEMENT firstname (#PCDATA)>
  <!ELEMENT lastname (#PCDATA)>
  <!ELEMENT school (#PCDATA)>
]>
<student>
  <firstname>Mark</firstname>
  <lastname>Wood</lastname>
  <school>Hills College</school>

```

</student>

In the above example,

- **!DOCTYPE student** indicates the beginning of the DTD declaration. And the **student** is the root element of the XML document.
- **!ELEMENT student** indicates the **student** element must contain **firstname**, **lastname** and **school** elements.
- **!ELEMENT firstname** indicates the **firstname** element is of type **#PCDATA** (Parsed Character Data).
- **!ELEMENT lastname** indicates the **lastname** element is of type **#PCDATA**.
- **!ELEMENT school** indicates the **school** element is of type **#PCDATA**.

Example of an external DTD:

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE student SYSTEM "student.dtd">
<student>
  <firstname>Mark</firstname>
  <lastname>Wood</lastname>
  <school>Hills College</school>
</student>
```

The DTD file content (student.dtd) as follows.

```
<!ELEMENT student (firstname,lastname,school)>
<!ELEMENT firstname (#PCDATA)>
<!ELEMENT lastname (#PCDATA)>
<!ELEMENT school (#PCDATA)>
```

XML DOM

What is DOM?

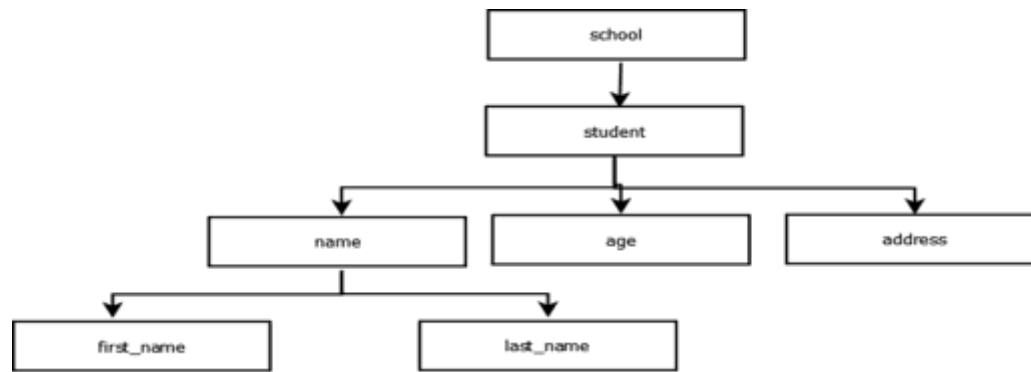
DOM stands for **Document Object Model**. It defines a standard manner of accessing and manipulating XML documents. DOM has a (hierarchical) tree structure.

Example of DOM

Let's consider the below XML document.

```
<?xml version="1.0" encoding="UTF-8" ?>
<school>
  <student>
    <name>
      <first_name>Alex</first_name>
      <last_name>Clarke</last_name>
    </name>
    <age>14</age>
    <address>No. 35, Flower Road, Leeds</address>
  </student>
</school>
```

The tree structure of the above XML file would look like the following image.



XML Validation

What are Well-formed XML Documents?

Well-formed XML documents are XML documents with correct syntax.

What are Valid XML Documents?

Valid XML documents are well-formed and also conform to the DTD rules.

XML Namespaces

Why Namespaces?

Namespaces help to avoid element name conflicts.

Namespace Declaration

Following shows the syntax for the namespace declaration.

```
<element xmlns:name="URL">
```

In the above declaration,

- The **xmlns** keyword indicates the beginning of the namespace.
- The **name** is the prefix of the namespace.
- The **URL** is the namespace identifier.

Examples of Namespaces

Following code segment shows an example of namespaces.

```
<?xml version="1.0" encoding="UTF-8" ?>
<abt:about xmlns:abt="https://www.guru99.com/about-us.html">
  <abt:founder>Krishna</abt:founder>
  <abt:vision>Fun and Free Education for ALL</abt:vision>
</abt:about>
```

XML Schemas

XML **schema** (also known as **XML schema definition** or **XSD**) use to describe the XML document structure. It is an alternative to DTD.

Why Schema is Important?

DTD is not powerful as schema as it is not extensible and flexible enough. So, it may not be suitable for some situations. In such a situation schema is important. The main purpose of using XML schema is to define the elements and attributes of an XML document.

How XML Schema is Different from DTD?

The following comparison shows how XSD (XML Schema) is different from DTD.

	DTD	XSD
Stands for	Document Type Definition	XML Schema Definition
Extensibility	Not extensible	Extensible
Control on XML structure	Less control	More control
Data types of support	Not supported	Supported
Namespace Support	Not supported	Supported

Following code segment shows an example of XML schema.

```

xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">
  <xs:element name = "employee">
    <xs:complexType>
      <xs:sequence>
        <xs:element name = "firstname" type = "xs:string" />
        <xs:element name = "lastname" type = "xs:string" />
        <xs:element name = "phone" type = "xs:int" />
      </xs:sequence>
    </xs:complexType>
  </xs:element>
</xs:schema>

```

Advantages of XML

Here, pros/benefits of XML:

- It made it easy to transport and share data.
- XML improves the exchange of data between various platforms.
- It is a markup language, which is a set of characters or/and symbols placed in a text document.
- XML indicates how the XML document should look after it is displayed.
- It simplifies the platform change process.
- It enhances data availability.
- It supports multilingual documents and Unicode.
- Provide relatively easy to learn and code.
- It is a markup language, which is a set of characters or/and symbols placed in a text document.
- It performs validation using DTD and Schema.
- Makes documents transportable across systems and applications. With the help of XML, you can exchange data quickly between different platforms.
- XML separates the data from HTML.

Disadvantages of XML

Here are the cons/drawback of using XML:

- XML requires a processing application.
- The XML syntax is similar to another alternative 'text-based' data transmission formats, which is sometimes confusing.
- No intrinsic data type support
- The XML syntax is redundant.
- Does not allow the user to create his tags.

AJAX is an acronym for **Asynchronous JavaScript and XML**. It is a group of inter-related technologies like JavaScript, DOM, XML, HTML/XHTML, CSS, XMLHttpRequest etc.

AJAX allows you to send and receive data asynchronously without reloading the web page. So it is fast.

AJAX allows you to send only important information to the server not the entire page. So only valuable data from the client side is routed to the server side. It makes your application interactive and faster.

Importance of AJAX in Web Development

Its significance lies in its ability to revolutionize the way web applications function and interact with users. Here are five key reasons why AJAX is crucial in web development:

1. Seamless User Experience

AJAX empowers web applications to update specific content sections without requiring complete page reloads. This results in a fluid and responsive user experience.

Users can interact with the website, and changes occur seamlessly, offering a dynamic and engaging interface. Whether it's submitting a form, browsing a product catalog, or engaging in discussions, AJAX ensures that user interactions are quicker, smoother, and highly responsive.

2. Swift Page Loading

By fetching data from the server without refreshing the entire page, AJAX substantially reduces loading times. This agility in content retrieval ensures that users access information swiftly, making for a more efficient and enjoyable experience.

Whether it's navigating through an online store, checking real-time stock quotes, or viewing live scores, AJAX eliminates the need for lengthy page reloads, resulting in near-instantaneous access to the desired content.

3. Granular Page Updates

The hallmark of AJAX is its ability to update only the portions of a web page that have changed. This feature underpins real-time functionality, such as notifications, live search results, and interactive forms.

Users can engage with the web application without disruptions. For example, when a user receives a new message in a chat application, only the message section updates, preserving the ongoing conversation and providing a seamless transition for the user.

4. Efficient Bandwidth Usage

AJAX is not just about speed; it's about efficiency. It sends and receives only the data required, minimizing the volume of data transmitted between clients and servers.

This frugality in bandwidth usage is invaluable for users with limited bandwidth or those on mobile devices. Whether it's checking email on a slow connection or using a data-intensive app on a mobile device, AJAX optimizes data transfer, ensuring that the user experience remains smooth and responsive.

5. Real-Time Interactions

AJAX is the cornerstone of real-time functionality, enabling chat systems, live updates, and collaborative applications. It fosters dynamic user engagement by allowing users to interact with each other and with the application in real-time.

Whether it's participating in a live webinar, receiving instant updates on social media, or collaborating on a shared document, AJAX ensures that users are at the forefront of the action, fostering interactivity that is essential in modern web applications.

In summary, AJAX's significance in web development lies in its capacity to elevate the user experience, making it a foundational technology for creating engaging and efficient web applications.

How AJAX Works

In the complex **web development process**, AJAX (Asynchronous JavaScript and XML) assumes the role of a conductor, directing the seamless flow of data between a web page and a server. Grasping the inner workings of AJAX unveils the enchantment that underlies dynamic and responsive web applications.

1. Client-Side Event

The performance begins with a client-side event, such as a user clicking a button, filling out a form, or scrolling down a page. This event triggers a JavaScript function that acts as the director of the show.

2. Asynchronous Request

The JavaScript function, now in the spotlight, initiates an asynchronous request to the server. This request, often made using the XMLHttpRequest (XHR) or Fetch API, is sent to a specific URL on the server, typically to fetch data or update information.

3. Server-Side Processing

On the server side, there's a team of handlers ready to process the request. These server-side scripts or APIs interpret the request, perform the necessary operations (like retrieving data from a database or processing form submissions), and prepare a response.

4. Data Exchange

The response from the server, which can be in the form of XML, JSON, or other data formats, is sent back to the web page. This is where the asynchronous part comes into play. The web page can continue to function independently while waiting for the server's response.

5. DOM Manipulation

Once the response arrives back at the web page, another JavaScript function takes center stage. This function is responsible for processing the data and dynamically updating the Document Object Model (DOM) of the web page.

It can add, remove, or modify elements on the page without requiring a full page reload. The result is a seamless and responsive user experience, where data is fetched and updated in the background, often imperceptible to the user.

Unlocking the mechanics of AJAX reveals the approach behind dynamic and responsive web applications. This streamlines the integration of technology and has grown to be an essential tool in the modern web developer's toolkit, ensuring that users encounter an effortless and captivating online journey.

Examples of AJAX in Real-World Applications

The real-world application of AJAX is a testament to its transformative power in delivering dynamic and responsive online experiences. Here are some notable instances where AJAX takes the spotlight:

1. Google Maps

Google Maps harnesses AJAX to redefine online mapping. When users zoom in, pan, or interact with the map, AJAX works behind the scenes to efficiently load and update map tiles. This creates a seamless, interactive mapping experience, allowing users to explore locations effortlessly.

2. Facebook

The social media, **Facebook**, relies extensively on AJAX to power its real-time interactions. Features like instant messaging, comment updates, and dynamic content loading in the news feed are driven by AJAX. Users receive instant notifications and can engage in conversations without the disruption of page refreshes.

3. Twitter

Twitter is a prime example of AJAX's role in delivering live content. As new tweets pour in, AJAX ensures they appear automatically without the need for manual page refreshes. This dynamic real-time feature keeps users up to date with the latest tweets and trending topics.

4. Gmail

Google's **Gmail** service employs AJAX for email composition and conversation views. Whether composing emails or navigating through message threads, AJAX offers a fluid, desktop-like experience without the need for complete page reloads. Users can manage their emails efficiently with minimal interruptions.

5. YouTube

YouTube leverages AJAX for video playback and content recommendations. When users select videos, AJAX seamlessly loads and plays the video. Simultaneously, the recommended videos on the sidebar update dynamically based on user interactions, enhancing content discovery and user engagement.

These real-world examples demonstrate how AJAX has become a cornerstone of modern web development, enriching user interactions and providing the dynamic, real-time experiences that users have come to expect.

AJAX is a technology that has seamlessly integrated into our daily digital lives, enhancing the way we access information and engage with web applications. Its influence continues to shape the future of web development.

AJAX - Technologies

AJAX is not a programming language or script language, but it combines multiple web-related technologies like HTML, XHTML, CSS, JavaScript, DOM, XML, XSLT and XMLHttpRequest object. Due to the combination of these technologies, the AJAX model allows web developers to create web applications that can dynamically interact with the user and can able to quickly make a background call to web servers to retrieve the required application data and then update the small portion of the web page without refreshing the whole web page.

AJAX does not use any new language to create dynamic web applications, it uses the technologies that are already present in the market. So makes it easier for the developers to

create a dynamic web application without learning or installing new technologies. Hence the web technologies used by the AJAX model are –

Javascript – It is a scripting language for HTML and the web application. It creates a connection between HTML, CSS, and XML. It is used to create client-side functionality. It also plays an important role in AJAX. It is also used to create AJAX applications or join all the AJAX operations together.

```
<script src = "myexample.js"></script>
```

XML or JSON – XML stands for extensible markup language whereas JSON stands for JavaScript Object Notation. Both JSON and XML are used on the client side to exchange data between the web server and the client.

```
<?xml version = "1.0">
```

```
<root>
```

```
  <child>
```

```
    //Statements
```

```
  </child>
```

```
</root>
```

XMLHttpRequest – It is used to perform asynchronous data exchange between a web browser and a web server. It is a javascript object that performs asynchronous operations.

```
variableName = new XMLHttpRequest();
```

HTML and CSS – HTML stands for hypertext markup language whereas CSS stands for cascading style sheets. HTML provides markup and style to the webpage text. Or we can say it provides a structure to the web page whereas CSS is used to create more interactive web pages. It provides various styling components that define the look of the web page. CSS is independent of HTML and can be used with any XML-based markup language.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  // Header of the web page
```

```
</head>
```

```
<body>
```

```
  // Body of the web page
```

```
</body>
```

```
</html>
```

DOM – AJAX also has a powerful tool known as DOM(Document Object Model). It is used to interact with and alter the webpage layout and content dynamically. Or we can say that DOM is used to create a logical representation of the elements that are used to markup HTML pages. It is provided by the web browser. It is not a part of JavaScript, but using JavaScript we can access the methods and the properties of DOM objects. Using DOM methods and properties we can create or modify HTML pages.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  // Header of the web page
```

```
</head>
```

```
<body>
```

```
<p></p>
<script></script>
</body>
</html>
```

AJAX - Action

Steps of AJAX Operation

- A client event occurs.
- An XMLHttpRequest object is created.
- The XMLHttpRequest object is configured.
- The XMLHttpRequest object makes an asynchronous request to the Webserver.
- The Webserver returns the result containing XML document.
- The XMLHttpRequest object calls the callback() function and processes the result.
- The HTML DOM is updated.

A Client Event Occurs

- A JavaScript function is called as the result of an event.
- Example – *validateUserId()* JavaScript function is mapped as an event handler to an *onkeyup* event on input form field whose id is set to "userid"
- `<input type = "text" size = "20" id = "userid" name = "id" onkeyup = "validateUserId();">`.

AJAX - XMLHttpRequest

In AJAX, XMLHttpRequest plays a very important role. XMLHttpRequest is used to exchange data to or from the web server in the background while the user/client working in the foreground and then update the part of the web page with the received data without reloading the whole page.

We can also say that XMLHttpRequest (XHR) can be used by various web browser scripting languages like JavaScript, JScript, VBScript, etc., to exchange XML data to or from the web server with the help of HTTP. Apart from XML, XMLHttpRequest can also fetch data in various formats like JSON, etc. It creates an asynchronous connection between the client side and the server side.

Syntax

```
variableName = new XMLHttpRequest()
```

Where using a new keyword along with XMLHttpRequest() constructor we can be able to create a new XMLHttpRequest object. This object must be created before calling the open() function to initialise it before calling send() function to send the request to the web server.

XMLHttpRequest Object Methods

XMLHttpRequest object has the following methods –

Sr.No.	Method Name & Description
1	new XMLHttpRequest() It is used to create an XMLHttpRequest() object
2	getAllResponseHeaders() It is used to get the header information
3	getResponseHeader() It is used to get the specific header information
4	open(method, url, async, user, psw)

	It is used to initialise the request parameters. Here, method: request type GET or POST or Other types url: file location async: for the asynchronous set to true or for synchronous set to false user: for optional user name psw: for optional password
5	send() It is used to send requests to the web server. It is generally used for GET requests.
6	send(string) It is used to send requests to the server. It is generally used for POST requests.
7	setRequestHeader() It is used to add key/value pair to the header

XMLHttpRequest Object Properties

XMLHttpRequest object has the following properties –

Sr.No.	Property Name & Description
1	onreadystatechange Set the callback function which handles request state changes.
2	readyState It is used to hold the status of XMLHttpRequest. It has the following values – • It represents the request is not initialise • It represents the server connection established • It represents the request received • It represents the request is in processing • It represents the request finished and the response is ready
3	responseText It is used to return the response data as a string
4	responseXML It is used to return the response data as XML data
5	Status It is used to return the status number of a request. For example – • 200: for OK • 403: for Forbidden • 404: for Not Found
6	StatusText It is used to return the status text. For example, OK, Not Found, etc.

Usage of XMLHttpRequest

After understanding the basic syntax, methods, and properties of XMLHttpRequest now we learn how to use XMLHttpRequest in real life. So to use XMLHttpRequest in your program first we need to follow the following major steps –

Step 1 – Create an object of XMLHttpRequest

```
var variableName = new XMLHttpRequest()
```

Step 2 – After creating XMLHttpRequest an object, we now have to define a callback function which will trigger after getting a response from the web server.

```
XMLHttpRequestObjectName.onreadystatechange = function(){  
    // Callback function body  
}
```

```
XMLHttpRequestObjectName.open(method, url, async)  
XMLHttpRequestObjectName.send()
```

Step 3 – Now we use open() and send() functions to send a request to the web server.

Example

In the below example, we are going to fetch data from the server. To fetch the data from the server we will click on the "Click Me" button. So when we click on the "Click Me" button, the displayDoc() function is called. Inside the displayDoc() function, we create an XMLHttpRequest object. Then we create a call-back function to handle the server response. Then we call the open() method of the XHR object to initialise the request with HTTP GET method and the server URL which is "<https://jsonplaceholder.typicode.com/todos>". Then we call send() function to send the request.

So when the server responds to the request, the "onreadystatechange" property calls the callback function with the current state of XMLHttpRequest object. If the "ready state" property is set to 4(that means the request is completed) and the "status" property is set to 200(that means the successful response), then the response data is extracted from the "responseText" property and display the HTML document with the help of "innerHTML" property of the sample element.

If we error is found during the request then the else statement present in the callback function will execute. So this is how we can fetch data from the server.

Open Compiler

```
<!DOCTYPE html>  
<html>  
<body>  
<script>  
    function displayDoc() {  
        // Creating XMLHttpRequest object  
        var myObj = new XMLHttpRequest();  
  
        // Creating a callback function  
        myObj.onreadystatechange = function() {  
            if (this.readyState == 4 && this.status == 200) {  
                document.getElementById("sample").innerHTML = this.responseText;  
            }else{  
                console.log("Error Found")  
            }  
        };  
        // Open the given file  
        myObj.open("GET", "https://jsonplaceholder.typicode.com/todos", true);  
  
        // Sending the request to the server  
        myObj.send();
```

```
}  
</script>  
<div id="sample">  
  <h2>Getting Data</h2>  
  <p>Please click on the button to fetch data</p>  
  <button type="button" onclick="displayDoc()">Click Me</button>  
</div>  
</body>  
</html>
```